Einer Barba Abdala 595839

Renata García Morales 612194

Título: Actividad 4: Single-Neuron Network With And WithoutPCA

Date: 06/03/2025

Materia: Inteligencia Artificial II

Declaramos que hemos realizado esta actividad con intégridad académica.

Dominio de aplicación y características generales

Este dataset pertenece al área médica, siendo que este evalúa el área de detección de evaluación de factores de riesgo asociados al cáncer cervical. Los datos fueron recolectados a partir del "Hospital Universitario de Caracas" en Venezuela e incluye una diversa cantidad de datos desde demografías, hábitos de vida y antecedentes médicos de los pacientes. El dataset contiene variables tanto enteras como booleanas, contiene datos faltantes que se manejarán durante el ejercicio y contiene un total de 36 variables y 858 pacientes.

El dataset está siendo hosteado en el sitio web UC Irvine Machine Learning Repository: https://archive.ics.uci.edu/dataset/383/cervical+cancer+risk+factors

Este dataset cuenta con una variable target de tipo bool, representando la ausencia o presencia de cáncer. Por ello, se tomará este dataset desde una perspectiva de un problema de clasificación.

Variables predictoras

Estas variables se pueden dividir en 5 categorías distintas.

1.- Existe un único tipo de variable que de información demográfica ya que se sabe que todas las pacientes son mujeres.

- Edad

2.- Por otro lado se encuentran las variables relacionadas a los antecedentes tanto sexuales como reproductivos.

- Número de parejas sexuales
- Edad de la primera relación sexual
- Número de embarazos

3.- Diferentes hábitos o elementos en el estilo de vida de la persona como lo es el consumo del tabaco al igual que el uso de métodos anticonceptivos.

- Fuma
- Años fumando
- Paquetes por año
- Uso de anticonceptivos hormonales
- Años de uso de anticonceptivos hormonales
- Uso de DIU
- Años de uso del DIU

4.- Historial de enfermades de transmisión sexual.

- Indicador general de ETS
- Número de ETS
- Número de diagnósticos de ETS
- Tiempo desde el primer diagnóstico de ETS
- Tiempo desde el último diagnóstico de ETS
- ETS: Condilomatosis
- ETS: Condilomatosis cervical
- ETS: Condilomatosis vaginal
- ETS: Condilomatosis vulvo-perineal
- ETS: Sífilis
- ETS: Enfermedad pélvica inflamatoria
- ETS: Herpes genital
- ETS: Molusco contagioso
- ETS: SIDA

- ETS: VIH
- ETS: Hepatitis B
- ETS: Virus del Papiloma Humano (VPH)

5.- Diagnósticos previos hechos al paciente.

- Dx (diagnóstico general previo)
- Dx:Cancer
- Dx:CIN
- Dx:HPV

Variable objetivo

Se eligió la variable de "Biopsy" o biopsia en español como la variable objetivo. Esta variable indica si el resultado del estudio confirma la presencia de tejido canceroso. Esto lo hace una variable binaria donde los resultados pueden ser ya sea 0 o 1. En caso de 0, el resultado es negativo, por lo que no hay cáncer. Un 1 significaría un resultado positivo, es decir, hay presencia de cáncer o al menos una anomalía significativa.

In [348…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.utils import class_weight
from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score, recall_score, f1_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

1. Antes de realizar la limpieza de los datos se carga el dataset. Para facilitar la limpieza se cambian todos los signos de interrogación a valores nulos para ser maniuplados más fácilmente en pasos próximos. Después, se utiliza head() para confirmar que se leyó el documento correctamente. También se usa shape para poder observar el tamaño del dataset.

In [349…

```
df = pd.read_csv("risk_factors_cervical_cancer.csv", header=0,na_values="?")
```

```
print(df.shape)
df.head()
```

```
(858, 36)
```

Out[349…

	Age	Number of sexual partners	First sexual intercourse	Num of pregnancies	Smokes	Smokes (years)	Smokes (packs/ year)	Hormonal Contraceptives	Hormonal Contraceptives (years)	IUD	…	STDs: Time since first diagnosis	STDs: Time since last diagnosis	Dx:Cancer	Dx:CIN	Dx:HPV	D
0	18	4.0	15.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	…	NaN	NaN	0	0	0	
1	15	1.0	14.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	…	NaN	NaN	0	0	0	
2	34	1.0	NaN	1.0	0.0	0.0	0.0	0.0	0.0	0.0	…	NaN	NaN	0	0	0	
3	52	5.0	16.0	4.0	1.0	37.0	37.0	1.0	3.0	0.0	…	NaN	NaN	1	0	1	
4	46	3.0	21.0	4.0	0.0	0.0	0.0	1.0	15.0	0.0	…	NaN	NaN	0	0	0	

5 rows × 36 columns

Se puede observar que la DataFrame ha sido generada correctamente, y se cuentan con 858 entradas, cada una con 36 variables.

2. Se utiliza df.columns para poder saber el nombre de cada una de las variables para facilitar el proceso futuro.

In [350…

```
df.columns
```

Out[350…

```
Index(['Age', 'Number of sexual partners', 'First sexual intercourse',
       'Num of pregnancies', 'Smokes', 'Smokes (years)', 'Smokes (packs/year)',
       'Hormonal Contraceptives', 'Hormonal Contraceptives (years)', 'IUD',
       'IUD (years)', 'STDs', 'STDs (number)', 'STDs:condylomatosis',
       'STDs:cervical condylomatosis', 'STDs:vaginal condylomatosis',
       'STDs:vulvo-perineal condylomatosis', 'STDs:syphilis',
       'STDs:pelvic inflammatory disease', 'STDs:genital herpes',
       'STDs:molluscum contagiosum', 'STDs:AIDS', 'STDs:HIV',
       'STDs:Hepatitis B', 'STDs:HPV', 'STDs: Number of diagnosis',
       'STDs: Time since first diagnosis', 'STDs: Time since last diagnosis',
       'Dx:Cancer', 'Dx:CIN', 'Dx:HPV', 'Dx', 'Hinselmann', 'Schiller',
       'Citology', 'Biopsy'],
      dtype='object')
```

Limpieza de datos

1. Antes de iniciar con la limpieza de datos se eliminan estas 3 variables las cuales servirían como otros casos de variables objetivo. Dado que se busca utilizar como variable objetivo Biopsy, se eliminan estas otras pruebas que podrían ocasionar fuga de datos, y empeorar el rendimiento del modelo.

In [351…

```
df = df.drop(columns=[
    "Hinselmann",
    "Schiller",
    "Citology",
])
```

2. Se observan cuales son los datos nulos de cada una de las variables para saber como manejarlos próximamente.

In [352…

```
missing =  df.isnull().sum() / len(df) * 100
missing.sort_values(ascending=False)
```

Out[352…

	0
STDs: Time since first diagnosis	91.724942
STDs: Time since last diagnosis	91.724942
IUD (years)	13.636364
IUD	13.636364
Hormonal Contraceptives	12.587413
Hormonal Contraceptives (years)	12.587413
STDs:HPV	12.237762
STDs (number)	12.237762
STDs	12.237762
STDs:vaginal condylomatosis	12.237762
STDs:vulvo-perineal condylomatosis	12.237762
STDs:HIV	12.237762
STDs:AIDS	12.237762
STDs:pelvic inflammatory disease	12.237762
STDs:syphilis	12.237762
STDs:cervical condylomatosis	12.237762
STDs:condylomatosis	12.237762
STDs:genital herpes	12.237762
STDs:Hepatitis B	12.237762
STDs:molluscum contagiosum	12.237762
Num of pregnancies	6.526807
Number of sexual partners	3.030303
Smokes (packs/year)	1.515152
Smokes (years)	1.515152
Smokes	1.515152
First sexual intercourse	0.815851

	0
Age	0.000000
STDs: Number of diagnosis	0.000000
Dx:Cancer	0.000000
Dx:CIN	0.000000
Dx:HPV	0.000000
Dx	0.000000
Biopsy	0.000000

dtype: float64

3. En las variables "STDs: Time since first diagnosis" y "STDs: Time since last diagnosis", más del 90% de los valores son nulos. Por ello, se decidió eliminar las columnas, ya que no aportan suficiente información valiosa como para justificar el mantenerlas.

In [353…

```
df = df.drop(columns=["STDs: Time since first diagnosis", "STDs: Time since last diagnosis"])
```

4. Las demás columnas con nulos que contaran con un porcentaje nulo mayor al 5% se imputaron por medio del uso de la mediana. Esto permite utilizar estas entradas restantes, con datos que podrían ser considerados los más "normales". Usar la mediana en lugar de la media permite evitar sesgos causados por outliers.

In [354…

```
cols_imputar = [
    "IUD", "IUD (years)", "Hormonal Contraceptives", "Hormonal Contraceptives (years)",
    "STDs:HPV", "STDs:AIDS", "STDs:Hepatitis B", "STDs:HIV", "STDs",
    "STDs:cervical condylomatosis", "STDs:vulvo-perineal condylomatosis", "STDs:syphilis",
    "STDs:pelvic inflammatory disease", "STDs:vaginal condylomatosis", "STDs:genital herpes",
    "STDs:molluscum contagiosum", "STDs:condylomatosis", "STDs (number)", "Num of pregnancies"
]

for col in cols_imputar:
    df[col] = df[col].fillna(df[col].median())
```

5. En el resto de variables que contenían menos del 5% de nulos se optó por eliminar las filas, ya que esto representa una pérdida mínima de información, mientras que también minimizamos el imputar más datos de los que ya hemos hecho en el paso pasado. Esto nos permite un buen balance entre mantener una cantidad útil de entradas y además una cantidad mínima de datos "generados".

In [355…

```
cols_dropna = ["Number of sexual partners", "Smokes", "Smokes (packs/year)", "Smokes (years)", "First sexual intercourse"]
```

```
df = df.dropna(subset=cols_dropna)
```

6. Se corrobora que una vez se realizó la limpieza ya no quedan más nulos en el dataset.

In [356…

```
missing =  df.isnull().sum() / len(df) * 100
missing.sort_values(ascending=False)
```

Out[356…

	0
Age	0.0
Number of sexual partners	0.0
First sexual intercourse	0.0
Num of pregnancies	0.0
Smokes	0.0
Smokes (years)	0.0
Smokes (packs/year)	0.0
Hormonal Contraceptives	0.0
Hormonal Contraceptives (years)	0.0
IUD	0.0
IUD (years)	0.0
STDs	0.0
STDs (number)	0.0
STDs:condylomatosis	0.0
STDs:cervical condylomatosis	0.0
STDs:vaginal condylomatosis	0.0
STDs:vulvo-perineal condylomatosis	0.0
STDs:syphilis	0.0
STDs:pelvic inflammatory disease	0.0
STDs:genital herpes	0.0
STDs:molluscum contagiosum	0.0
STDs:AIDS	0.0
STDs:HIV	0.0
STDs:Hepatitis B	0.0
STDs:HPV	0.0
STDs: Number of diagnosis	0.0

	0
Dx:Cancer	0.0
Dx:CIN	0.0
Dx:HPV	0.0
Dx	0.0
Biopsy	0.0

dtype: float64

7. Se ve la forma final para verificar que los cambios fueron hechos exitosamente y que aún se conserve un número de entradas significante.

In [357…

```
print(df.shape)
df.head(30)
```

```
(814, 31)
```

Out[357…

	Age	Number of sexual partners	First sexual intercourse	Num of pregnancies	Smokes	Smokes (years)	Smokes (packs/ year)	Hormonal Contraceptives	Hormonal Contraceptives (years)	IUD	…	STDs:AIDS	STDs:HIV	STDs:Hepatitis B	STDs:HPV
0	18	4.0	15.0	1.0	0.0	0.000000	0.00	0.0	0.00	0.0	…	0.0	0.0	0.0	0.0
1	15	1.0	14.0	1.0	0.0	0.000000	0.00	0.0	0.00	0.0	…	0.0	0.0	0.0	0.0
3	52	5.0	16.0	4.0	1.0	37.000000	37.00	1.0	3.00	0.0	…	0.0	0.0	0.0	0.0
4	46	3.0	21.0	4.0	0.0	0.000000	0.00	1.0	15.00	0.0	…	0.0	0.0	0.0	0.0
5	42	3.0	23.0	2.0	0.0	0.000000	0.00	0.0	0.00	0.0	…	0.0	0.0	0.0	0.0
6	51	3.0	17.0	6.0	1.0	34.000000	3.40	0.0	0.00	1.0	…	0.0	0.0	0.0	0.0
7	26	1.0	26.0	3.0	0.0	0.000000	0.00	1.0	2.00	1.0	…	0.0	0.0	0.0	0.0
8	45	1.0	20.0	5.0	0.0	0.000000	0.00	0.0	0.00	0.0	…	0.0	0.0	0.0	0.0
9	44	3.0	15.0	2.0	1.0	1.266973	2.80	0.0	0.00	0.0	…	0.0	0.0	0.0	0.0
10	44	3.0	26.0	4.0	0.0	0.000000	0.00	1.0	2.00	0.0	…	0.0	0.0	0.0	0.0
11	27	1.0	17.0	3.0	0.0	0.000000	0.00	1.0	8.00	0.0	…	0.0	0.0	0.0	0.0
12	45	4.0	14.0	6.0	0.0	0.000000	0.00	1.0	10.00	1.0	…	0.0	0.0	0.0	0.0
13	44	2.0	25.0	2.0	0.0	0.000000	0.00	1.0	5.00	0.0	…	0.0	0.0	0.0	0.0
14	43	2.0	18.0	5.0	0.0	0.000000	0.00	0.0	0.00	1.0	…	0.0	0.0	0.0	0.0
15	40	3.0	18.0	2.0	0.0	0.000000	0.00	1.0	15.00	0.0	…	0.0	0.0	0.0	0.0
16	41	4.0	21.0	3.0	0.0	0.000000	0.00	1.0	0.25	0.0	…	0.0	0.0	0.0	0.0
17	43	3.0	15.0	8.0	0.0	0.000000	0.00	1.0	3.00	0.0	…	0.0	0.0	0.0	0.0
18	42	2.0	20.0	2.0	0.0	0.000000	0.00	1.0	7.00	1.0	…	0.0	0.0	0.0	0.0
19	40	2.0	27.0	2.0	0.0	0.000000	0.00	0.0	0.00	1.0	…	0.0	0.0	0.0	0.0
20	43	2.0	18.0	4.0	0.0	0.000000	0.00	1.0	15.00	0.0	…	0.0	0.0	0.0	0.0
21	41	3.0	17.0	4.0	0.0	0.000000	0.00	1.0	10.00	0.0	…	0.0	0.0	0.0	0.0
22	40	1.0	18.0	1.0	0.0	0.000000	0.00	1.0	0.25	0.0	…	0.0	0.0	0.0	0.0
23	40	1.0	20.0	2.0	0.0	0.000000	0.00	1.0	15.00	0.0	…	0.0	0.0	0.0	0.0
24	40	3.0	15.0	3.0	0.0	0.000000	0.00	1.0	3.00	0.0	…	0.0	0.0	0.0	0.0

	Age	Number of sexual partners	First sexual intercourse	Num of pregnancies	Smokes	Smokes (years)	Smokes (packs/ year)	Hormonal Contraceptives	Hormonal Contraceptives (years)	IUD	...	STDs:AIDS	STDs:HIV	STDs:Hepatitis B	STDs:HPV
25	44	3.0	19.0	1.0	0.0	0.000000	0.00	0.0	0.00	0.0	...	0.0	0.0	0.0	0.0
26	39	5.0	23.0	2.0	0.0	0.000000	0.00	0.0	0.00	1.0	...	0.0	0.0	0.0	0.0
27	39	2.0	17.0	4.0	0.0	0.000000	0.00	0.0	0.00	0.0	...	0.0	0.0	0.0	0.0
28	37	3.0	24.0	1.0	1.0	3.000000	0.04	0.0	0.00	0.0	...	0.0	0.0	0.0	0.0
29	37	6.0	26.0	1.0	0.0	0.000000	0.00	1.0	0.25	0.0	...	0.0	0.0	0.0	0.0
30	41	3.0	17.0	3.0	0.0	0.000000	0.00	1.0	22.00	1.0	...	0.0	0.0	0.0	0.0

30 rows × 31 columns

Como se puede observar, se eliminaron solamente 44 entradas, aproximadamente un 5% de los datos originales. A cambio de esto, la DataFrame ahora está limpia sin datos nulos.

Análisis exploratorio de datos

1. Describe se utiliza para poder analizar las estadísticas descriptivas de las variables numéricas permitiendo observar elementos esenciales como el promedio, mínimo, máximo, etc.

In [358…

```
df.describe()
```

Out[358…

	Age	Number of sexual partners	First sexual intercourse	Num of pregnancies	Smokes	Smokes (years)	Smokes (packs/ year)	Hormonal Contraceptives	Hormonal Contraceptives (years)	IUD	…	STDs:AIDS	STDs:HIV	S
count	814.000000	814.000000	814.000000	814.000000	814.000000	814.000000	814.000000	814.000000	814.000000	814.000000	…	814.0	814.000000	
mean	26.852580	2.512285	17.013514	2.265356	0.147420	1.224076	0.458960	0.685504	2.094268	0.099509	…	0.0	0.019656	
std	8.582306	1.606941	2.840329	1.410156	0.354742	4.081365	2.257969	0.464600	3.636691	0.299528	…	0.0	0.138901	
min	13.000000	1.000000	10.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	…	0.0	0.000000	
25%	20.000000	2.000000	15.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	…	0.0	0.000000	
50%	25.000000	2.000000	17.000000	2.000000	0.000000	0.000000	0.000000	1.000000	0.500000	0.000000	…	0.0	0.000000	
75%	32.000000	3.000000	18.000000	3.000000	0.000000	0.000000	0.000000	1.000000	2.445550	0.000000	…	0.0	0.000000	
max	84.000000	28.000000	32.000000	11.000000	1.000000	37.000000	37.000000	1.000000	30.000000	1.000000	…	0.0	1.000000	

8 rows × 31 columns

Refiriéndose a las edades, se puede observar que la edad promedio es de apróximadamente 27 años. Existen pacientes bastante jóvenes teniendo 13 años, y algunos de los pacientes mayores tienen hasta 84. La desviación es de 8.5, lo cual es moderado. Finalmente, se ve que los pacientes se concentran entre los 20 y 32 años, ya que se muestra que el 50% de los valores caen en este rango.

Otras variables nos permiten saber que solo el 14% de los pacientes fuman, y existe una gran cantidad de pacientes que usan anticonceptivos hormonales siendo estos el 68%.

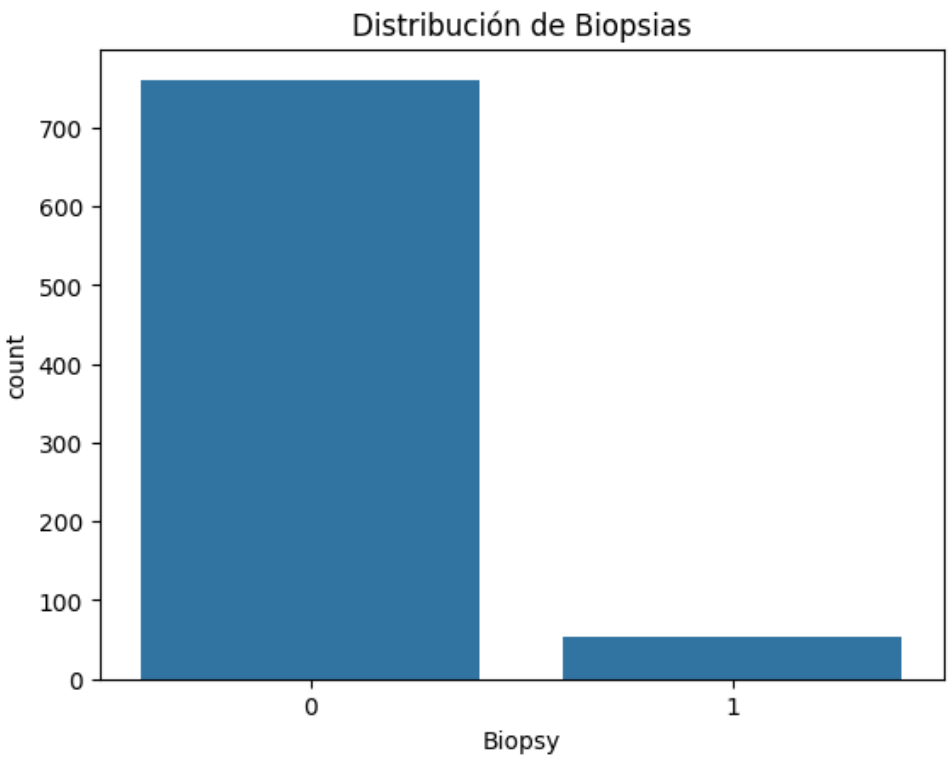
La variable "Number of sexual partners" cuenta con una media de 2.51 y una mediana de 2, pero tiene valores máximos de hasta 28. Estos outliers podrían ser relevantes.

Se pueden observar númericamente las variables binarias, representadas por un min de 0 y un max de 1. Varias de estas variables cuentan con means muy cercanos al 0, lo cual indica un sesgo hacia valores de 0 más frecuentes.

2. Con esta gráfica se puede observar la distribución de la variable objetivo.

In [359…

```
sns.countplot(x='Biopsy', data=df)
plt.title('Distribución de Biopsias')
plt.show()
,
```



Out[359…

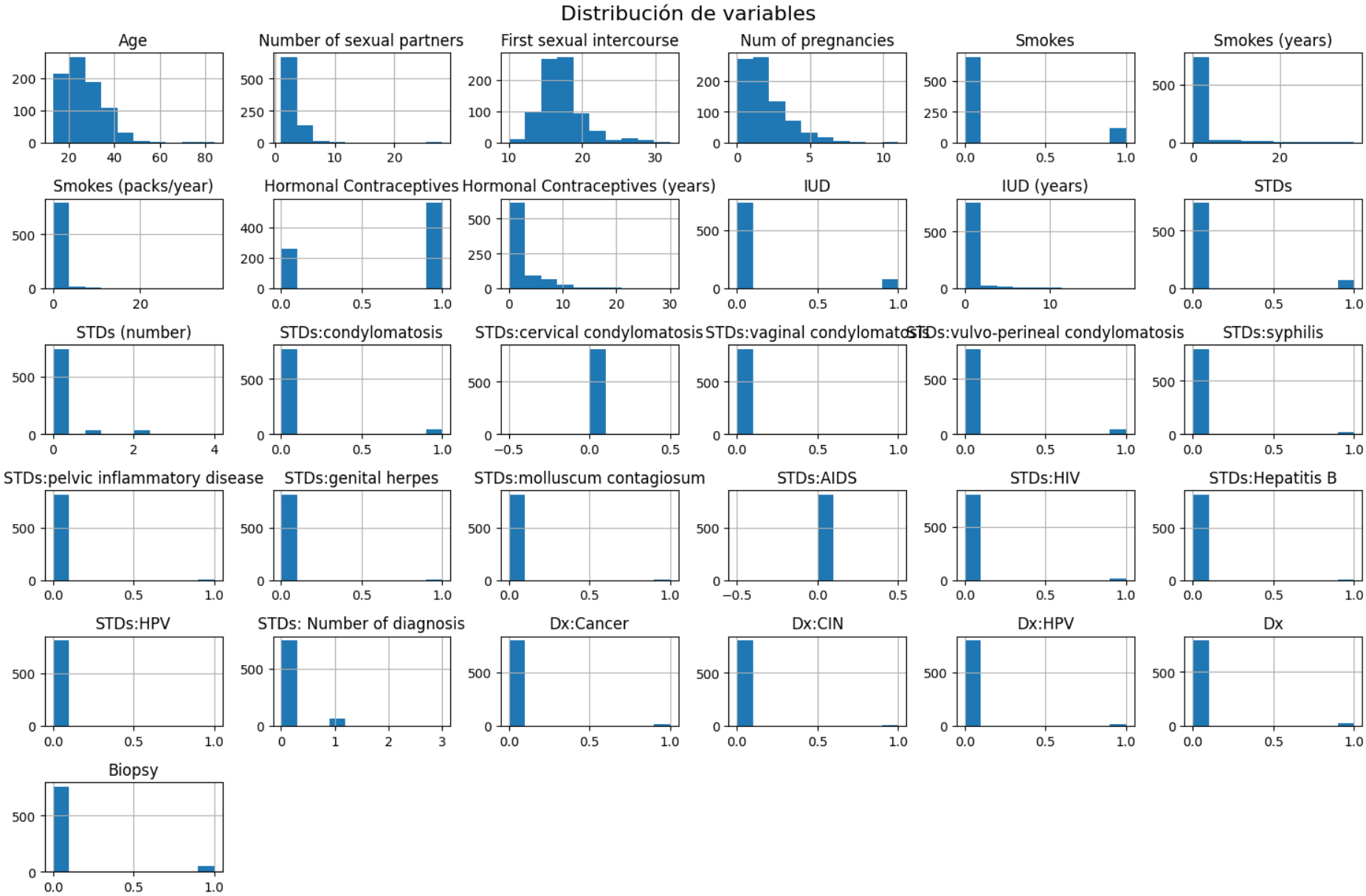
```
''
```

La gráfica permite obervar que existe un problema de desbalance basante grande donde los casos negativos son bastante más numerosos que los positivos. Esto podría ser riesgoso en la creación del modelo, ya que el modelo podría adoptar este sesgo y tener una dificultad para predecir valores de 1.

3. Se realiza un histrograma el cual grafica la distribución de cada variable.

In [360…

```
df.hist(figsize=(15,10))
plt.suptitle("Distribución de variables", fontsize=16)
plt.tight_layout()
plt.show()
```



Se puede observar que en edad, la distribución se inclina más hacia los jóvenes.

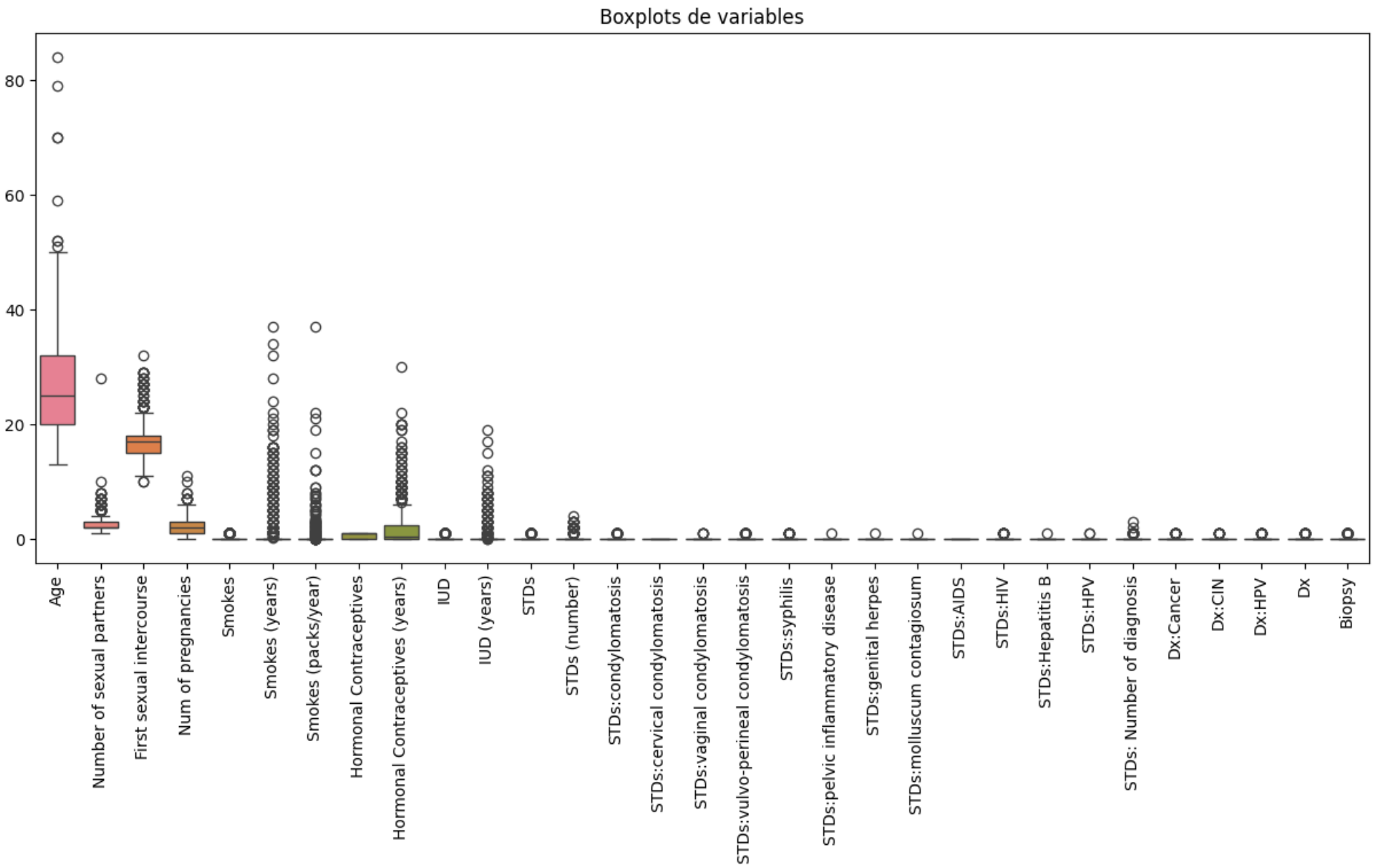
Por otro lado, en la variable de parejas sexuales, se muestra un sesgo positivo donde hay muchos valores pequeños y pocos valores realmente altos. Esto se ajusta a la observación hecha anteriormente mediante el describe.

Con respecto al tabaco, la mayoría no fuma, pero existen ciertos outliers que fuman cantidades significativas.

4. Se realizan boxplots los cuales muestran la mediana, los cuartiles y los valores que se consideran outliers esto de cada variable.

In [361…

```
plt.figure(figsize=(15,6))
sns.boxplot(data=df)
plt.xticks(rotation=90) # Rotar texto para que sea vertical.
plt.title("Boxplots de variables")
plt.show()
```

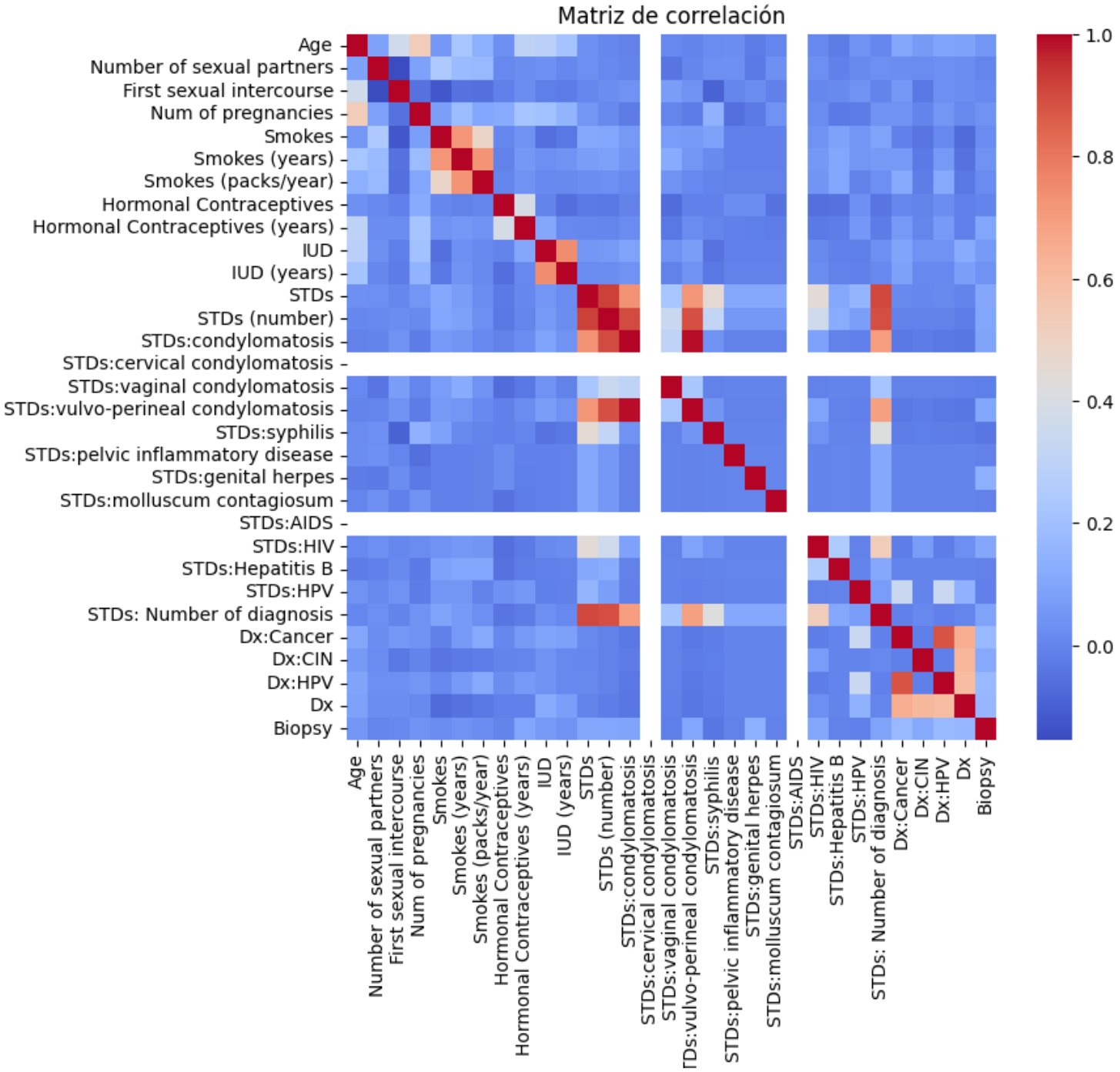


Esta visualización confirma los patrones notados anteriormente, donde los outliers son más comunes en las variables de número de parejas sexuales, años/cantidad que se fuma, y además en número de embarazos.

5. La matriz de correlación ayuda a calcular la correlación de Pearson entre todas las variables. Esto se visualiza por medio del heatmap.

In [362…

```
plt.figure(figsize=(8,7))
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
plt.title("Matriz de correlación")
plt.show()
```


Matriz de correlación
Age
Number of sexual partners
First sexual intercourse
Num of pregnancies
Smokes
Smokes (years)
Smokes (packs/year)
Hormonal Contraceptives
Hormonal Contraceptives (years)
IUD
IUD (years)
STDs
STDs (number)
STDs:condylomatosis
STDs:cervical condylomatosis
STDs:vaginal condylomatosis
STDs:vulvo-perineal condylomatosis
STDs:syphilis
STDs:pelvic inflammatory disease
STDs:genital herpes
STDs:molluscum contagiosum
STDs:AIDS
STDs:HIV
STDs:Hepatitis B
STDs:HPV
STDs: Number of diagnosis
Dx:Cancer
Dx:CIN
Dx:HPV
Dx
Biopsy
1.0
0.8
0.6
0.4
0.2
0.0

La matriz muestra que la mayor parte de las variables tienen correlaciones bajas, pero existen algunas relacionadas con diagnósticos médicos que muestran correlaciones más altas entre sí. Esto tiene sentido debido a que ciertos diagnósticos se relacionan de una manera clínica.

6. Finalmente, se realizaron promedios por clase agrupados por la variable objetivo la cual es Biopsy.

In [363…
```
df.groupby("Biopsy").mean()
```

Out[363…

Biopsy	Age	Number of sexual partners	First sexual intercourse	Num of pregnancies	Smokes	Smokes (years)	Smokes (packs/ year)	Hormonal Contraceptives	Hormonal Contraceptives (years)	IUD	…	STDs:molluscum contagiosum	STDs:AIDS	STDs:H
0	26.735874	2.512484	17.005256	2.250986	0.145861	1.182826	0.448069	0.685940	2.000321	0.094612	…	0.001314	0.0	0.0157
1	28.528302	2.509434	17.132075	2.471698	0.169811	1.816358	0.615343	0.679245	3.443208	0.169811	…	0.000000	0.0	0.0754

2 rows × 30 columns

Se dividió el dataset en grupo 0 y 1 de biopsy. Se muestran diferencias pequeñas entre los resultados por ejemplo Age con 26.7 y 28.5 o Smokes con 0.145 y 0.169. Hay una diferencia más grande en Hormonal Contraceptives la cual es de 2 y 3.44. Las diferencias son en su gran mayoría pequeñas lo cual podría sugerir que las características no separan de manera clara las clases.

Modelo de una neurona

1. Se inicia separando las variables predictoras de la variable objetivo. La "y" siendo Biopsy y la "X" siendo todas las demás variables restantes, consdierando que varias ya fueron eliminadas durante la limpieza.

In [364…
```
y = df["Biopsy"]
X = df.drop(columns=["Biopsy"])
```

2. Después, se separa el 20% de los datos para la prueba y se utiliza stratify=y para mantener la proporción de clases en cada subconjunto. Esto se considera debido a que el dataset muestra desbalance. Luego se divide el conjunto de entrenamiento.

In [365…
```
X_train_full, X_test, y_train_full, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full, test_size=0.2, random_state=42, stratify=y_train_full)

print("Training size:", X_train.shape)
print("Validation size:", X_val.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (520, 30)
Validation size: (131, 30)
Testing size: (163, 30)
```

Los resultados muestran que 520 de los datos se usan para entrenar, otros 131 para validar en el entrenamiento y 163 para el desempeño final.

3. Después, se normalizan las variables para que tengan la media de 0 y la desviación estándar de 1. Esto permite evitar problemas futuros donde la escala de los datos numéricos erróneamente sesga el entrenamiento.

In [366…

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

Siguiente, se realiza el cálculo de pesos por desbalance. Esto se debe a que el dataset tiene un desbalance que debe ser manejado si queremos que el modelo aprenda los patrones de casos tanto positivos como negativos.

In [367…

```
class_weights = class_weight.compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)

class_weights = dict(enumerate(class_weights)) # Convertir lista a diccionario.

print("Class Weights:", class_weights)
```

```
Class Weights: {0: np.float64(0.5349794238683128), 1: np.float64(7.647058823529412)}
```

Mediante este proceso, podemos compensar el desbalance de clases, castigando de manera más estricta errores en la clase minoritaria.

4. Se define el modelo de una sola neurona y una sola capa. El model.compile nos permite describir las métricas a seguir durante el entrenamiento, así como el optimizador a utilizar.

In [368…

```
# Definir red neuronal, con semilla.
modelo = tf.keras.Sequential([
    tf.keras.Input(shape=(X_train.shape[1],)), # Definir shape de la entrada.
```

```python
    tf.keras.layers.Dense(                    # Definir un modelo de 1 neurona.
        1,
        activation='sigmoid',
        kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)
    )
])

# Configurar entrenamiento.
modelo.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Mostrar arquitectura.
modelo.summary()
```

Model: "sequential_16"

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 1)	31

Total params: 31 (124.00 B)

Trainable params: 31 (124.00 B)

Non-trainable params: 0 (0.00 B)

Se cuenta con 31 parámetros, siendo 30 features y la neurona de salida con bias, sumando los 31.

5. Se realiza el entrenamiento de la neurona usando los datos de entrenamiento. Se tienen 100 épocas en las cuales se ajustan los 31 parámetros. Como se ve se, se utiliza el class weight para penalizar el modelo cuando este se equivoca en la clase minoritaria.

In [369…

```python
history = modelo.fit(
    X_train,
    y_train,
    epochs=100, # Definir hiperparámetros básicos.
    batch_size=32,
    validation_data=(X_val, y_val),
    class_weight=class_weights,
    verbose=1 # Mostrar detalles.
)
```

```
Epoch 1/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.4233 - loss: 1.0849 - val_accuracy: 0.3969 - val_loss: 0.9305
Epoch 2/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4286 - loss: 1.1800 - val_accuracy: 0.3893 - val_loss: 0.9234
Epoch 3/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4146 - loss: 1.1135 - val_accuracy: 0.3969 - val_loss: 0.9181
Epoch 4/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4167 - loss: 1.1465 - val_accuracy: 0.3969 - val_loss: 0.9133
Epoch 5/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4065 - loss: 1.0688 - val_accuracy: 0.3893 - val_loss: 0.9080
Epoch 6/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3886 - loss: 0.9679 - val_accuracy: 0.3817 - val_loss: 0.9034
Epoch 7/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.3765 - loss: 0.9759 - val_accuracy: 0.3817 - val_loss: 0.8988
Epoch 8/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.3895 - loss: 1.1755 - val_accuracy: 0.3893 - val_loss: 0.8942
Epoch 9/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4241 - loss: 0.8853 - val_accuracy: 0.3893 - val_loss: 0.8901
Epoch 10/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4063 - loss: 1.0353 - val_accuracy: 0.3893 - val_loss: 0.8851
Epoch 11/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3737 - loss: 0.8883 - val_accuracy: 0.3969 - val_loss: 0.8802
Epoch 12/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4132 - loss: 1.0908 - val_accuracy: 0.3893 - val_loss: 0.8760
Epoch 13/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4281 - loss: 1.0071 - val_accuracy: 0.3664 - val_loss: 0.8723
Epoch 14/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4151 - loss: 0.8805 - val_accuracy: 0.3511 - val_loss: 0.8688
Epoch 15/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.3971 - loss: 0.8781 - val_accuracy: 0.3511 - val_loss: 0.8662
Epoch 16/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4424 - loss: 0.8983 - val_accuracy: 0.3435 - val_loss: 0.8634
Epoch 17/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4580 - loss: 0.8837 - val_accuracy: 0.3511 - val_loss: 0.8589
Epoch 18/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4168 - loss: 0.8359 - val_accuracy: 0.3359 - val_loss: 0.8573
Epoch 19/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4390 - loss: 0.8340 - val_accuracy: 0.3435 - val_loss: 0.8562
Epoch 20/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4279 - loss: 0.8755 - val_accuracy: 0.3511 - val_loss: 0.8554
Epoch 21/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4268 - loss: 0.8984 - val_accuracy: 0.3511 - val_loss: 0.8528
Epoch 22/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4441 - loss: 0.8203 - val_accuracy: 0.3511 - val_loss: 0.8494
Epoch 23/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4193 - loss: 0.6862 - val_accuracy: 0.3588 - val_loss: 0.8453
Epoch 24/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4436 - loss: 0.6691 - val_accuracy: 0.3588 - val_loss: 0.8438
```

```
Epoch 25/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4209 - loss: 0.8207 - val_accuracy: 0.3588 - val_loss: 0.8447
Epoch 26/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4633 - loss: 0.7599 - val_accuracy: 0.3664 - val_loss: 0.8426
Epoch 27/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4459 - loss: 0.7778 - val_accuracy: 0.3664 - val_loss: 0.8395
Epoch 28/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4594 - loss: 0.7654 - val_accuracy: 0.3664 - val_loss: 0.8373
Epoch 29/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4576 - loss: 0.7532 - val_accuracy: 0.3664 - val_loss: 0.8353
Epoch 30/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4895 - loss: 0.7909 - val_accuracy: 0.3664 - val_loss: 0.8328
Epoch 31/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4451 - loss: 0.7749 - val_accuracy: 0.3664 - val_loss: 0.8302
Epoch 32/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4616 - loss: 0.7378 - val_accuracy: 0.3588 - val_loss: 0.8273
Epoch 33/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5044 - loss: 0.7372 - val_accuracy: 0.3588 - val_loss: 0.8256
Epoch 34/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4680 - loss: 0.8284 - val_accuracy: 0.3740 - val_loss: 0.8232
Epoch 35/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4649 - loss: 0.7594 - val_accuracy: 0.3817 - val_loss: 0.8207
Epoch 36/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4844 - loss: 0.7348 - val_accuracy: 0.3969 - val_loss: 0.8189
Epoch 37/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5022 - loss: 0.8194 - val_accuracy: 0.4046 - val_loss: 0.8176
Epoch 38/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5406 - loss: 0.6394 - val_accuracy: 0.4122 - val_loss: 0.8154
Epoch 39/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.5271 - loss: 0.7693 - val_accuracy: 0.4351 - val_loss: 0.8154
Epoch 40/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5673 - loss: 0.6584 - val_accuracy: 0.4427 - val_loss: 0.8134
Epoch 41/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5537 - loss: 0.6513 - val_accuracy: 0.4427 - val_loss: 0.8120
Epoch 42/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5683 - loss: 0.7347 - val_accuracy: 0.4885 - val_loss: 0.8095
Epoch 43/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5963 - loss: 0.7113 - val_accuracy: 0.4962 - val_loss: 0.8096
Epoch 44/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6105 - loss: 0.6708 - val_accuracy: 0.5191 - val_loss: 0.8078
Epoch 45/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6108 - loss: 0.7136 - val_accuracy: 0.5344 - val_loss: 0.8069
Epoch 46/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6631 - loss: 0.6545 - val_accuracy: 0.5573 - val_loss: 0.8065
Epoch 47/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6970 - loss: 0.6529 - val_accuracy: 0.5649 - val_loss: 0.8061
Epoch 48/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6612 - loss: 0.7459 - val_accuracy: 0.5649 - val_loss: 0.8038
```

```
Epoch 49/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.7105 - loss: 0.6466 - val_accuracy: 0.5802 - val_loss: 0.8022
Epoch 50/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7079 - loss: 0.6389 - val_accuracy: 0.5878 - val_loss: 0.7992
Epoch 51/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6802 - loss: 0.6873 - val_accuracy: 0.5725 - val_loss: 0.7979
Epoch 52/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6979 - loss: 0.6995 - val_accuracy: 0.5802 - val_loss: 0.7964
Epoch 53/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7304 - loss: 0.6749 - val_accuracy: 0.5802 - val_loss: 0.7945
Epoch 54/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7411 - loss: 0.6191 - val_accuracy: 0.5878 - val_loss: 0.7902
Epoch 55/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7000 - loss: 0.7282 - val_accuracy: 0.5954 - val_loss: 0.7878
Epoch 56/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6972 - loss: 0.6293 - val_accuracy: 0.5954 - val_loss: 0.7856
Epoch 57/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6986 - loss: 0.7511 - val_accuracy: 0.5954 - val_loss: 0.7857
Epoch 58/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7304 - loss: 0.6301 - val_accuracy: 0.6031 - val_loss: 0.7838
Epoch 59/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7304 - loss: 0.6903 - val_accuracy: 0.6183 - val_loss: 0.7831
Epoch 60/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7620 - loss: 0.5942 - val_accuracy: 0.6336 - val_loss: 0.7816
Epoch 61/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7561 - loss: 0.5388 - val_accuracy: 0.6260 - val_loss: 0.7782
Epoch 62/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7619 - loss: 0.6502 - val_accuracy: 0.6260 - val_loss: 0.7769
Epoch 63/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7543 - loss: 0.7130 - val_accuracy: 0.6336 - val_loss: 0.7747
Epoch 64/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7548 - loss: 0.6462 - val_accuracy: 0.6260 - val_loss: 0.7726
Epoch 65/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7575 - loss: 0.6268 - val_accuracy: 0.6489 - val_loss: 0.7720
Epoch 66/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7372 - loss: 0.6473 - val_accuracy: 0.6641 - val_loss: 0.7703
Epoch 67/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7657 - loss: 0.6631 - val_accuracy: 0.6641 - val_loss: 0.7694
Epoch 68/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7515 - loss: 0.6037 - val_accuracy: 0.6641 - val_loss: 0.7672
Epoch 69/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7651 - loss: 0.6087 - val_accuracy: 0.6718 - val_loss: 0.7672
Epoch 70/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7442 - loss: 0.6297 - val_accuracy: 0.6718 - val_loss: 0.7656
Epoch 71/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7744 - loss: 0.6807 - val_accuracy: 0.6641 - val_loss: 0.7654
Epoch 72/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7719 - loss: 0.6547 - val_accuracy: 0.6641 - val_loss: 0.7627
```

```
Epoch 73/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7562 - loss: 0.6178 - val_accuracy: 0.6718 - val_loss: 0.7623
Epoch 74/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7823 - loss: 0.5743 - val_accuracy: 0.6718 - val_loss: 0.7608
Epoch 75/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7741 - loss: 0.5736 - val_accuracy: 0.6718 - val_loss: 0.7584
Epoch 76/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7443 - loss: 0.6325 - val_accuracy: 0.6718 - val_loss: 0.7590
Epoch 77/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7495 - loss: 0.6332 - val_accuracy: 0.6718 - val_loss: 0.7593
Epoch 78/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7429 - loss: 0.6132 - val_accuracy: 0.6718 - val_loss: 0.7588
Epoch 79/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7748 - loss: 0.5507 - val_accuracy: 0.6718 - val_loss: 0.7569
Epoch 80/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7581 - loss: 0.5491 - val_accuracy: 0.6718 - val_loss: 0.7556
Epoch 81/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7680 - loss: 0.6266 - val_accuracy: 0.6718 - val_loss: 0.7550
Epoch 82/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7833 - loss: 0.5158 - val_accuracy: 0.6718 - val_loss: 0.7519
Epoch 83/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7858 - loss: 0.5634 - val_accuracy: 0.6718 - val_loss: 0.7504
Epoch 84/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7700 - loss: 0.6076 - val_accuracy: 0.6718 - val_loss: 0.7510
Epoch 85/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7593 - loss: 0.6198 - val_accuracy: 0.6718 - val_loss: 0.7505
Epoch 86/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7926 - loss: 0.5917 - val_accuracy: 0.6718 - val_loss: 0.7482
Epoch 87/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7685 - loss: 0.6424 - val_accuracy: 0.6718 - val_loss: 0.7470
Epoch 88/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7894 - loss: 0.5629 - val_accuracy: 0.6718 - val_loss: 0.7471
Epoch 89/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7542 - loss: 0.5260 - val_accuracy: 0.6718 - val_loss: 0.7439
Epoch 90/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7701 - loss: 0.6282 - val_accuracy: 0.6718 - val_loss: 0.7436
Epoch 91/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8013 - loss: 0.5885 - val_accuracy: 0.6870 - val_loss: 0.7436
Epoch 92/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7676 - loss: 0.5969 - val_accuracy: 0.6870 - val_loss: 0.7426
Epoch 93/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7908 - loss: 0.5796 - val_accuracy: 0.6870 - val_loss: 0.7407
Epoch 94/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7947 - loss: 0.6062 - val_accuracy: 0.6947 - val_loss: 0.7404
Epoch 95/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7597 - loss: 0.6196 - val_accuracy: 0.6947 - val_loss: 0.7391
Epoch 96/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7915 - loss: 0.6210 - val_accuracy: 0.6947 - val_loss: 0.7386
```

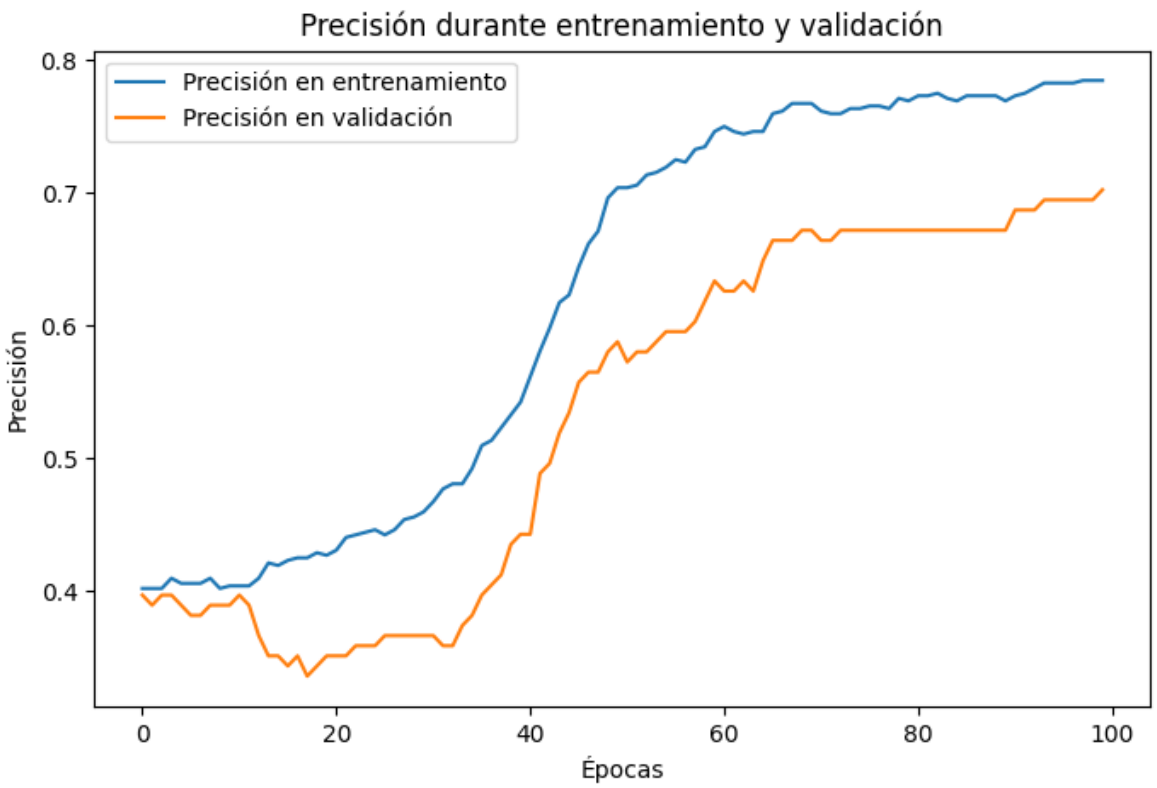
```
Epoch 97/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7825 - loss: 0.6084 - val_accuracy: 0.6947 - val_loss: 0.7371
Epoch 98/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7701 - loss: 0.6544 - val_accuracy: 0.6947 - val_loss: 0.7372
Epoch 99/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7646 - loss: 0.7170 - val_accuracy: 0.6947 - val_loss: 0.7370
Epoch 100/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7821 - loss: 0.6255 - val_accuracy: 0.7023 - val_loss: 0.7366
```

Los resultados demuestran que el accuracy inicia aproximadamente en 0.6 y empieza a aumentar poco a poco hasta llegar a valores cercanos a 0.8 en el entrenamiento, mientras que en validación comienza alrededor de 0.6 y sube gradualmente hasta aproximadamente 0.7. Al mismo tiempo, la pérdida disminuye, pasando de valores cercanos a 0.85 en validación a alrededor de 0.70 al final del entrenamiento. Esto indica que el modelo sí está aprendiendo durante las épocas, ya que mejora su capacidad de clasificación y reduce su error, aunque el desempeño no es completamente ideal.

6. En esta sección se grafica la evolución del accuracy durante el entrenamiento ya que se almacenan los valores registrados de cada época. Se grafican los valores de entrenamiento y los de validación.

In [370…

```python
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Precisión en entrenamiento')
plt.plot(history.history['val_accuracy'], label='Precisión en validación')
plt.legend()
plt.title("Precisión durante entrenamiento y validación")
plt.xlabel("Épocas")
plt.ylabel("Precisión")
plt.show()
```



Aquí se observa que las 2 curvas aumentan poco a poco y se mantienen cercanas. Esto da a entender que el modelo mejora poco a poco y no solo memoriza los datos. Sin emabrgo, el accuracy no llega a valores altos, y el accuracy de validación se aplana alrededor de un valor de 0.73, mientras que el valor de accuracy de training continua incrementando levemente. Esto es un indicio de overfitting, donde ambas curvas se comienzan a alejar la una de la otra.

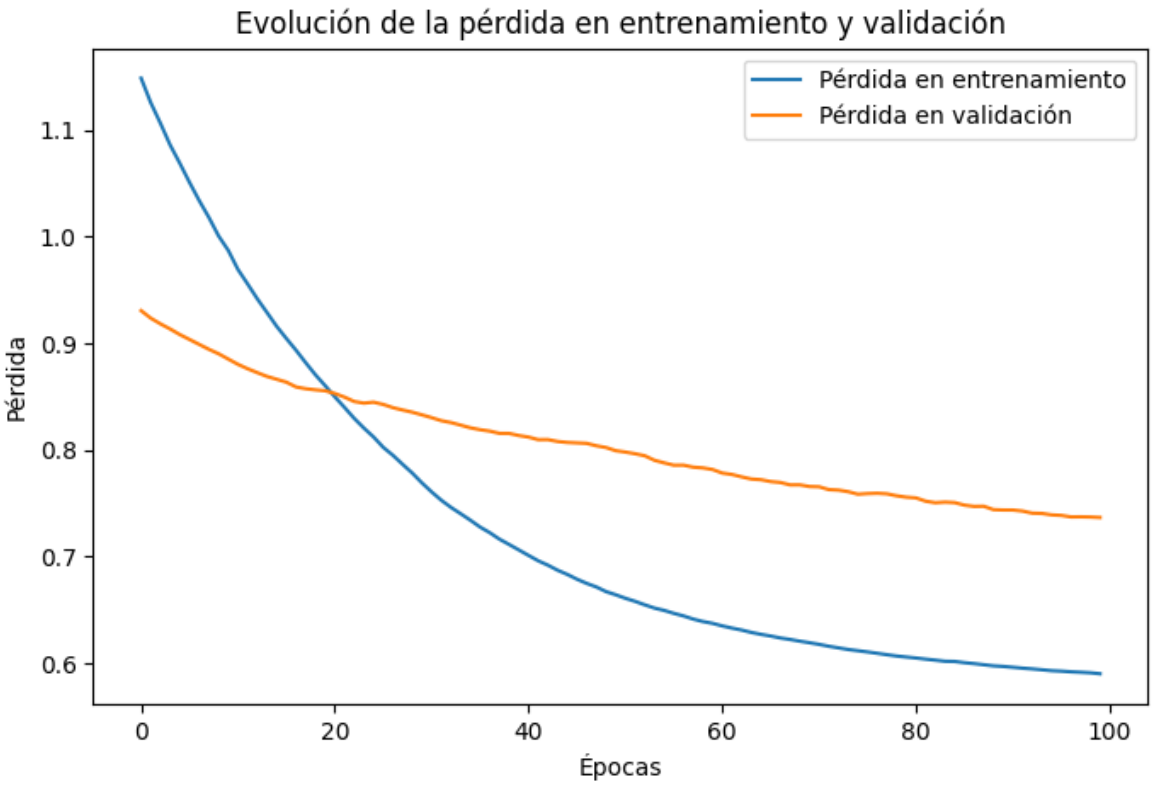
Hasta el momento, el modelo generaliza de manera aceptable, donde la accuracy de validación si incrementa a la par que la de entrenamiento, hasta el valor mencionado anteriormente.

7. Con el próximo código se busca graficar la evolución de la pérdida del modelo durante el entrenamiento. Se usan los valores guardados en history, donde la pérdida representa el error del modelo en el conjunto de entrenamiento y val_loss el error en el conjunto de validación.

In [371…

```
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Pérdida en entrenamiento')
plt.plot(history.history['val_loss'], label='Pérdida en validación')
plt.legend()
plt.title("Evolución de la pérdida en entrenamiento y validación")
plt.xlabel("Épocas")
```

```
plt.ylabel("Pérdida")
plt.show()
```



En la gráfica se muestra que tanto la pérdida de entrenamiento como la de validación disminuyen de una manera gradual a través de las épocas, lo que nos da a entender que el modelo está aprendiendo de manera correcta. A pesar de esto, la pérdida de la validación se mantiene un poco más alta que la de entrenamiento pero de todas maneras ambas siguen una tendencia similar y no se observa que la pérdida de validación aumente. Esto sugiere que el modelo tiene un ajuste razonablemente bueno y no presenta señales claras de overfitting, aunque aún existe margen de mejora en su capacidad de generalización.

8. En esta parte se usa el modelo entrenado para predecir las clases de los datos de prueba. Primero calcula las probabilidades de cada predicción y luego las convierte en valores binarios siendo 0 o 1 usando un umbral de 0.5 para obtener la clasificación final.

In [372…

```
y_pred_prob = modelo.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int)
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
```

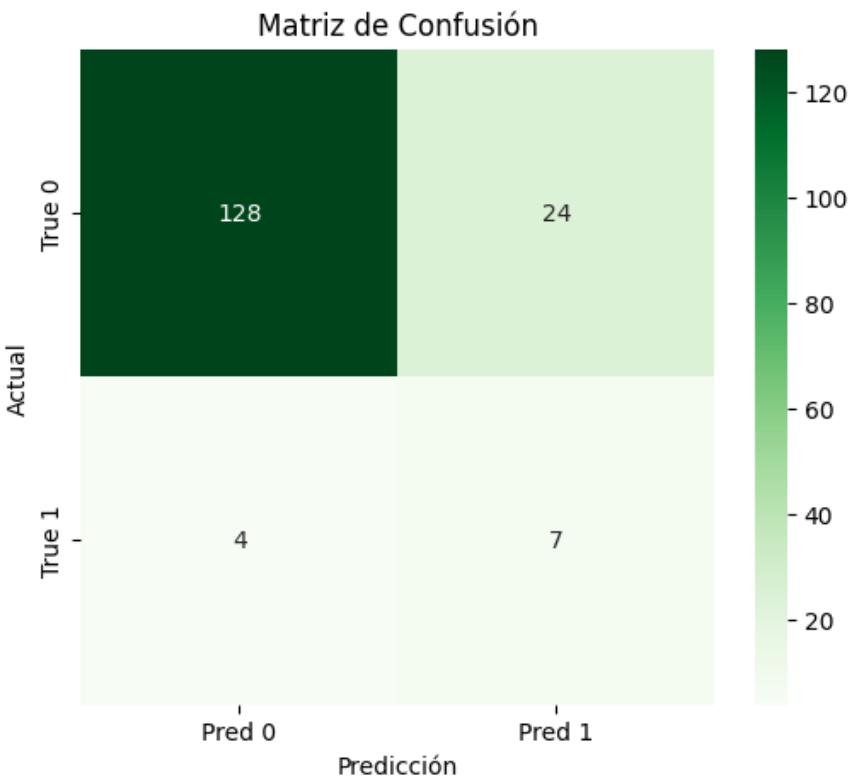
9. Después se calcula la matriz de confusión comparando el y_test con el y_pred. La función de confusion_matrix genera una tabla que muestra cuántas predicciones fueron correctas o incorrectas para cada clase.

In [373…

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Greens',
    xticklabels=['Pred 0','Pred 1'],
    yticklabels=['True 0','True 1']
)

plt.xlabel('Predicción')
plt.ylabel('Actual')
plt.title('Matriz de Confusión')
plt.show()
```



Aquí se ve que el modelo predijo de manera correcta 126 casos negativos y 5 casos positivos. Sin embargo, 26 casos negativos fueron clasificados incorrectamente como positivos, y 6 casos positivos fueron clasificados como negativos. Esto muestra que el modelo funciona mejor identificando la clase negativa que la positiva lo cual tiene sentido con el desbalance analizado anteriormente.

En nuestro caso, se podría decir que el valor más importante es el Recall, algo común en el área médica. Esta métrica define la cantidad de 1s que fueron correctamente identificados. El "asustar" a un paciente con una diagnosis falsa que luego puede ser analizada a profundidad para corregirla representa menos consecuencias, en comparación a permitir que un paciente enfermo piense que está sano, y se vaya sin tratamiento.

10. Finalmente, se calculan diversas métricas de evaluación del modelo usando las predicciones las cuales son y_pred y los valores reales siendo estos y_test. Se obtienen la precisión, el recall, la especificidad y el puntaje F1, que ayudan a medir qué tan bien el modelo clasifica los datos.

In [374…

```
# Calcular métricas.
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

# Extraer valores para matriz.
```

```
tn, fp, fn, tp = cm.ravel()
especificidad = tn / (tn + fp)

# Calcular F1 score.
f1 = f1_score(y_test, y_pred)

print("Precisión:", precision)
print("Sensibilidad (Recall):", recall)
print("Especificidad:", especificidad)
print("Puntaje F1:", f1)
```

```
Precisión: 0.22580645161290322
Sensibilidad (Recall): 0.6363636363636364
Especificidad: 0.8421052631578947
Puntaje F1: 0.3333333333333333
```

Los resultados muestran que la precisión es baja con un resultado de casi 0.2, lo que muestra que muchas de las predicciones positivas del modelo son incorrectas. El recall es 0.6, que significa que el modelo logra detectar cerca del 45% de los casos positivos reales. La especificidad es alta con alrededor de 0.8, lo que indica que el modelo identifica de manera correcta la mayoría de los casos negativos. Para terminar, el F1 score es bajo con alrededor de 0.3, lo que muestra un desempeño limitado del modelo para equilibrar precisión y recall.

Para nuestros propósitos, el performance del modelo es impresionante, considerando que solo se usa 1 capa y una neurona. Sin embargo, para aplicaciones médicas reales, una precisión tan baja del 0.2 sería inaceptable, incluso con un recall relativamente alto. Es claro que el modelo contiene áreas de oportunidad.

11. Al terminar se guarda el modelo entrenado en un archivo para poder usarlo después y load_model carga ese modelo guardado nuevamente para poder reutilizarlo sin tener que volver a entrenarlo.

In [375…

```
modelo.save("single_neuron_antes_de_pca.keras")
```

In [376…

```
modelo_cargado = tf.keras.models.load_model("single_neuron_antes_de_pca.keras")
```

Es importante el poder guardar y cargar modelos ya entrenados. Un entrenamiento es un cálculo de valores, que luego pueden usarse para calcular predicciones. Este cálculo suele llegar a ser la parte más tardada de la creación de un modelo de IA. Por ello, el tener el conocimiento de como guardar este cálculo, y cargarlo en otro momento, puede representar un enorme ahorro de tiempo.

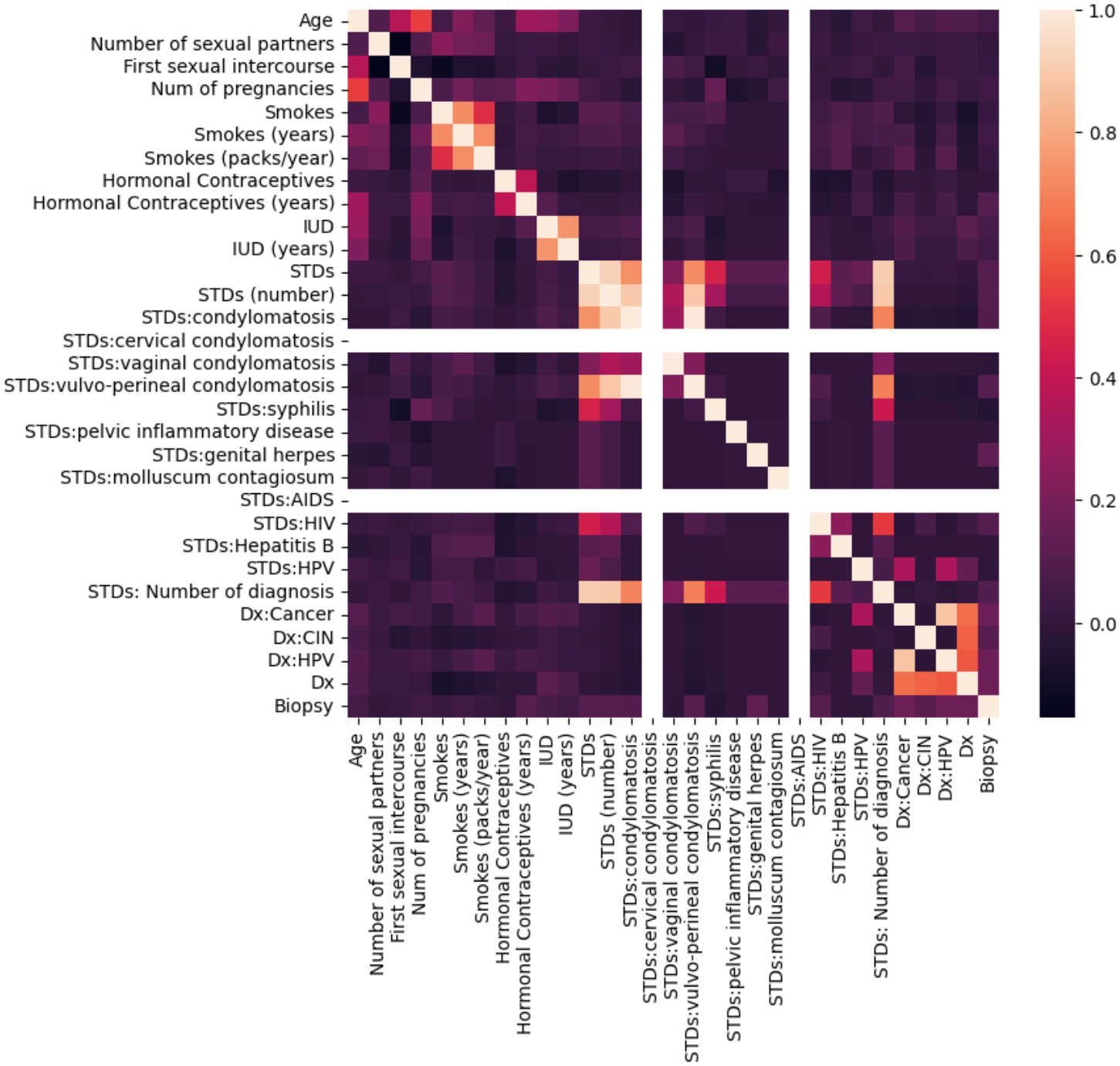
Implementación PCA

1. Primero, podemos utilizar una matriz de correlación para entender la relación entre nuestras variables.

In [377…

```
df_temp = df.copy()
df_temp = df_temp.corr(numeric_only=True)
```

```python
plt.figure(figsize=(8,7))
sns.heatmap(df_temp, fmt=".2f")
plt.show()
```


Age
Number of sexual partners
First sexual intercourse
Num of pregnancies
Smokes
Smokes (years)
Smokes (packs/year)
Hormonal Contraceptives
Hormonal Contraceptives (years)
IUD
IUD (years)
STDs
STDs (number)
STDs:condylomatosis
STDs:cervical condylomatosis
STDs:vaginal condylomatosis
STDs:vulvo-perineal condylomatosis
STDs:syphilis
STDs:pelvic inflammatory disease
STDs:genital herpes
STDs:molluscum contagiosum
STDs:AIDS
STDs:HIV
STDs:Hepatitis B
STDs:HPV
STDs: Number of diagnosis
Dx:Cancer
Dx:CIN
Dx:HPV
Dx
Biopsy
1.0
0.8
0.6
0.4
0.2
0.0

Podemos observar que existen varias variabels correlacionadas, significando que estas muestran información similar. Esto era de esperarse, pues existen variables como IUD y IUD (years), que pueden ser tomadas como maneras de representar las mismas tendencias con unidades diferentes.

Esto significa que nuestro data set es un buen caso donde usar PCA, ya que este proceso tomará estas variables con información similar y podrá reducir la dimensionalidad de nuestros datos.

2. Para comenzar con el proceso de PCA, debemos estandarizar los datos, removiendo sesgos por escalas de unidades. Podemos lograr esto con la función StandardScaler. Debemos también remover la variable Biopsy para evitar fuga de datos.

In [378…

```
df_x = df.drop(columns=['Biopsy'])
```

In [379…

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_x)
df_scaled
```

Out[379…

```
array([[-1.03212616e+00,  9.26374747e-01, -7.09337397e-01, ...,
        -1.05736107e-01, -1.46047894e-01, -1.70520138e-01],
       [-1.38189747e+00, -9.41674082e-01, -1.06162577e+00, ...,
        -1.05736107e-01, -1.46047894e-01, -1.70520138e-01],
       [ 2.93194871e+00,  1.54905769e+00, -3.57049025e-01, ...,
        -1.05736107e-01,  6.84706893e+00, -1.70520138e-01],
       ...,
       [-2.15993095e-01, -3.18991139e-01, -4.76065367e-03, ...,
        -1.05736107e-01, -1.46047894e-01, -1.70520138e-01],
       [ 7.16730403e-01, -3.18991139e-01,  2.46125795e+00, ...,
        -1.05736107e-01, -1.46047894e-01, -1.70520138e-01],
       [ 2.50368654e-01, -3.18991139e-01,  1.05210446e+00, ...,
        -1.05736107e-01, -1.46047894e-01, -1.70520138e-01]])
```

3. El siguiente paso es el cómputo de la matriz de covarianza.

In [380…

```
import numpy as np

cov_matrix = np.cov(df_scaled, rowvar=False)
print(cov_matrix)
```

```
[[ 1.00123001e+00  8.75539061e-02  3.68833648e-01  5.35540772e-01
   5.28652364e-02  2.21547427e-01  1.31869009e-01  3.31283108e-02
   2.99483791e-01  2.83104283e-01  2.15217683e-01  3.02744264e-02
   5.59361493e-03 -8.30341628e-03  0.00000000e+00  9.41253812e-03
  -5.87243528e-03  2.05649255e-02  2.51677513e-02 -2.80546996e-02
   6.03543197e-04  0.00000000e+00  1.07014234e-02 -2.80546996e-02
   4.14078886e-02  6.42336539e-03  1.03802854e-01  6.21630731e-02
   9.47770542e-02  8.60181742e-02]
 [ 8.75539061e-02  1.00123001e+00 -1.55048264e-01  8.49697174e-02
   2.40938283e-01  1.77583562e-01  1.73493885e-01  1.01363834e-02
   2.48634856e-02  3.19953630e-02  4.91079734e-03  2.80141315e-02
   1.08202192e-02 -1.78645639e-03  0.00000000e+00 -4.43497445e-02
  -1.29067234e-05  2.83398583e-02  3.25293432e-02 -3.30665743e-02
   3.25293432e-02  0.00000000e+00  2.65027991e-02 -1.12012685e-02
   1.50905099e-02  2.86925254e-02  2.29837764e-02  1.75015630e-02
   2.83398583e-02  2.41160694e-02]
 [ 3.68833648e-01 -1.55048264e-01  1.00123001e+00 -5.50128049e-02
  -1.24207531e-01 -5.26703834e-02 -5.58833903e-02 -1.35698684e-02
   2.26185903e-02 -1.75076956e-02 -2.51701948e-02  7.47091789e-03
   2.40667227e-02  3.80650179e-02  0.00000000e+00  7.40253054e-02
   4.24908774e-02 -9.46339073e-02 -1.67168781e-04  2.45738108e-02
  -1.25376586e-02  0.00000000e+00  5.56817585e-03  1.22033210e-02
   3.47740093e-02 -2.83559224e-03  5.38483722e-02 -3.36554750e-02
   2.96063674e-02  2.27226648e-02]
 [ 5.35540772e-01  8.49697174e-02 -5.50128049e-02  1.00123001e+00
   7.67047669e-02  1.80494585e-01  9.95983132e-02  1.18293962e-01
   2.19276959e-01  2.05573002e-01  1.51760814e-01  5.15980390e-02
   1.25171693e-02 -2.81903402e-02  0.00000000e+00 -7.66666901e-04
  -2.74546282e-02  1.43365113e-01 -5.64449196e-02 -3.15283445e-02
   4.32213809e-02  0.00000000e+00  1.73174278e-02 -3.15283445e-02
  -2.69857346e-02  3.64680887e-02  3.96053623e-02 -3.24023943e-03
   5.18123918e-02  1.52581140e-02]
 [ 5.28652364e-02  2.40938283e-01 -1.24207531e-01  7.67047669e-02
   1.00123001e+00  7.22591261e-01  4.89718716e-01  5.52615307e-03
   4.13153427e-02 -5.72678753e-02 -3.73424743e-02  1.07285747e-01
   1.01454668e-01  5.97299154e-02  0.00000000e+00  6.99733851e-02
   6.27534631e-02  8.47694708e-02 -1.46015673e-02 -1.46015673e-02
  -1.46015673e-02  0.00000000e+00  4.10212153e-02  8.44457308e-02
   4.94176832e-02  8.96114096e-02 -1.22802468e-02 -4.40218269e-02
   1.19821826e-02 -7.09938001e-02]
 [ 2.21547427e-01  1.77583562e-01 -5.26703834e-02  1.80494585e-01
   7.22591261e-01  1.00123001e+00  7.27016402e-01 -6.62751135e-03
   5.41963662e-02  2.90632818e-02  3.93942164e-02  7.40238558e-02
   8.39421751e-02  4.75156849e-02  0.00000000e+00  1.16882999e-01
   4.97483401e-02  1.62899970e-02 -1.05380030e-02 -1.05380030e-02
  -1.05380030e-02  0.00000000e+00  5.30379647e-02  1.01378310e-01
   5.20909535e-02  6.57623118e-02  5.73406077e-02 -3.17707090e-02
   6.00124390e-02 -5.12364780e-02]
```

```
[ 1.31869009e-01  1.73493885e-01 -5.58833903e-02  9.95983132e-02
  4.89718716e-01  7.27016402e-01  1.00123001e+00  2.64247921e-03
  4.31366605e-02  8.14718492e-03  1.61726503e-02  2.55641079e-02
  2.95229710e-02  8.71505381e-03  0.00000000e+00  4.18130625e-02
  9.95885535e-03 -3.00795248e-03 -7.14187619e-03 -7.14187619e-03
 -7.14187619e-03  0.00000000e+00  4.73421107e-02  1.01785119e-01
 -8.34474571e-03  2.78360150e-02  1.11905355e-01 -2.15318281e-02
  1.13861575e-01 -3.47242813e-02]
[ 3.31283108e-02  1.01363834e-02 -1.35698684e-02  1.18293962e-01
  5.52615307e-03 -6.62751135e-03  2.64247921e-03  1.00123001e+00
  3.90777572e-01 -4.65313925e-03 -6.15859129e-02 -3.12590783e-02
 -3.70296832e-02 -9.47434836e-03  0.00000000e+00 -6.59933998e-02
 -1.33923753e-02 -1.21075087e-02  2.37843600e-02  2.37843600e-02
 -5.18424722e-02  0.00000000e+00 -5.66411771e-02 -5.18424722e-02
  3.36568700e-02 -4.44109086e-02  6.41788992e-03 -4.29493572e-03
  2.49432885e-02 -1.22554484e-02]
[ 2.99483791e-01  2.48634856e-02  2.26185903e-02  2.19276959e-01
  4.13153427e-02  5.41963662e-02  4.31366605e-02  3.90777572e-01
  1.00123001e+00  1.05557929e-01  8.55460797e-03  5.75699465e-03
  1.99030911e-03  1.68758412e-02  0.00000000e+00 -3.57031074e-02
  1.87626631e-02 -1.42607370e-03 -1.29877857e-02 -1.78185868e-02
 -2.02339874e-02  0.00000000e+00 -3.04951681e-02 -2.02339874e-02
  5.33990851e-02 -2.62997918e-02  6.06757904e-02  2.10885381e-03
  6.34448140e-02 -1.41501443e-02]
[ 2.83104283e-01  3.19953630e-02 -1.75076956e-02  2.05573002e-01
 -5.72678753e-02  2.90632818e-02  8.14718492e-03 -4.65313925e-03
  1.05557929e-01  1.00123001e+00  7.47902381e-01  5.02490871e-02
  5.81261258e-02  8.95437062e-02  0.00000000e+00  3.53722705e-02
  7.36519910e-02 -4.86093352e-02 -1.16729092e-02 -1.16729092e-02
 -1.16729092e-02  0.00000000e+00  1.20730225e-02 -1.16729092e-02
 -1.65181484e-02  2.81112744e-02  9.50653157e-02  4.33990137e-02
  3.75954553e-02  1.16829514e-01]
[ 2.15217683e-01  4.91079734e-03 -2.51701948e-02  1.51760814e-01
 -3.73424743e-02  3.93942164e-02  1.61726503e-02 -6.15859129e-02
  8.55460797e-03  7.47902381e-01  1.00123001e+00  2.05298426e-02
  2.12649837e-02  3.62927522e-02  0.00000000e+00 -7.95132449e-03
  3.50616097e-02 -3.63103753e-02 -8.71947155e-03 -8.71947155e-03
 -8.71947155e-03  0.00000000e+00  2.23430048e-02 -8.71947155e-03
 -1.23387857e-02  1.11113178e-02  8.00728821e-02  8.15343758e-03
  1.02429277e-02  7.97764153e-02]
[ 3.02744264e-02  2.80141315e-02  7.47091789e-03  5.15980390e-02
  1.07285747e-01  7.40238558e-02  2.55641079e-02 -3.12590783e-02
  5.75699465e-03  5.02490871e-02  2.05298426e-02  1.00123001e+00
  9.22259674e-01  7.33063240e-01  0.00000000e+00  2.20857897e-01
  7.23815082e-01  4.59008520e-01  1.10225017e-01  1.10225017e-01
  1.10225017e-01  0.00000000e+00  4.45024573e-01  1.10225017e-01
  1.55977671e-01  9.08875351e-01  1.28973977e-02  6.94506978e-03
  1.28973977e-02 -3.05844435e-03]
```

```
[ 5.59361493e-03  1.08202192e-02  2.40667227e-02  1.25171693e-02
  1.01454668e-01  8.39421751e-02  2.95229710e-02 -3.70296832e-02
  1.99030911e-03  5.81261258e-02  2.12649837e-02  9.22259674e-01
  1.00123001e+00  9.01243346e-01  0.00000000e+00  3.46158732e-01
  8.91904301e-01  3.15834078e-01  5.62644726e-02  5.62644726e-02
  5.62644726e-02  0.00000000e+00  3.61546159e-01  1.22833195e-01
  7.96189615e-02  8.95216056e-01 -1.02967599e-02 -8.76641186e-03
 -1.02967599e-02 -2.19554654e-02]
[-8.30341628e-03 -1.78645639e-03  3.80650179e-02 -2.81903402e-02
  5.97299154e-02  4.75156849e-02  8.71505381e-03 -9.47434836e-03
  1.68758412e-02  8.95437062e-02  3.62927522e-02  7.33063240e-01
  9.01243346e-01  1.00123001e+00  0.00000000e+00  3.01651404e-01
  9.88598724e-01  4.36609457e-02 -8.19038999e-03 -8.19038999e-03
 -8.19038999e-03  0.00000000e+00  8.70988445e-02 -8.19038999e-03
 -1.15900908e-02  6.99101830e-01 -3.41071283e-02 -2.46929609e-02
 -3.41071283e-02 -3.98222258e-02]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00]
[ 9.41253812e-03 -4.43497445e-02  7.40253054e-02 -7.66666901e-04
  6.99733851e-02  1.16882999e-01  4.18130625e-02 -6.59933998e-02
 -3.57031074e-02  3.53722705e-02 -7.95132449e-03  2.20857897e-01
  3.46158732e-01  3.01651404e-01  0.00000000e+00  1.00123001e+00
  2.25078182e-01 -1.02758237e-02 -2.46760745e-03 -2.46760745e-03
 -2.46760745e-03  0.00000000e+00 -9.96276511e-03 -2.46760745e-03
 -3.49187211e-03  2.16259463e-01 -1.02758237e-02 -7.43951562e-03
 -1.02758237e-02 -1.19976730e-02]
[-5.87243528e-03 -1.29067234e-05  4.24908774e-02 -2.74546282e-02
  6.27534631e-02  4.97483401e-02  9.95885535e-03 -1.33923753e-02
  1.87626631e-02  7.36519910e-02  3.50616097e-02  7.23815082e-01
  8.91904301e-01  9.88598724e-01  0.00000000e+00  2.25078182e-01
  1.00123001e+00  4.49829821e-02 -8.08706191e-03 -8.08706191e-03
 -8.08706191e-03  0.00000000e+00  8.88939494e-02 -8.08706191e-03
 -1.14438729e-02  6.89831815e-01 -3.36768406e-02 -2.43814402e-02
 -3.36768406e-02 -3.93198377e-02]
[ 2.05649255e-02  2.83398583e-02 -9.46339073e-02  1.43365113e-01
  8.47694708e-02  1.62899970e-02 -3.00795248e-03 -1.21075087e-02
 -1.42607370e-03 -4.86093352e-02 -3.63103753e-02  4.59008520e-01
  3.15834078e-01  4.36609457e-02  0.00000000e+00 -1.02758237e-02
  4.49829821e-02  1.00123001e+00 -5.12842356e-03 -5.12842356e-03
 -5.12842356e-03  0.00000000e+00  4.12589420e-02 -5.12842356e-03
 -7.25715071e-03  4.20525796e-01 -2.13562236e-02 -1.54615302e-02
 -2.13562236e-02 -2.49347395e-02]
```

```
[ 2.51677513e-02  3.25293432e-02 -1.67168781e-04 -5.64449196e-02
 -1.46015673e-02 -1.05380030e-02 -7.14187619e-03  2.37843600e-02
 -1.29877857e-02 -1.16729092e-02 -8.71947155e-03  1.10225017e-01
  5.62644726e-02 -8.19038999e-03  0.00000000e+00 -2.46760745e-03
 -8.08706191e-03 -5.12842356e-03  1.00123001e+00 -1.23152523e-03
 -1.23152523e-03  0.00000000e+00 -4.97218332e-03 -1.23152523e-03
 -1.74271179e-03  1.07930046e-01 -5.12842356e-03 -3.71288845e-03
 -5.12842356e-03 -5.98775832e-03]
[-2.80546996e-02 -3.30665743e-02  2.45738108e-02 -3.15283445e-02
 -1.46015673e-02 -1.05380030e-02 -7.14187619e-03  2.37843600e-02
 -1.78185868e-02 -1.16729092e-02 -8.71947155e-03  1.10225017e-01
  5.62644726e-02 -8.19038999e-03  0.00000000e+00 -2.46760745e-03
 -8.08706191e-03 -5.12842356e-03 -1.23152523e-03  1.00123001e+00
 -1.23152523e-03  0.00000000e+00 -4.97218332e-03 -1.23152523e-03
 -1.74271179e-03  1.07930046e-01 -5.12842356e-03 -3.71288845e-03
 -5.12842356e-03 -5.98775832e-03]
[ 6.03543197e-04  3.25293432e-02 -1.25376586e-02  4.32213809e-02
 -1.46015673e-02 -1.05380030e-02 -7.14187619e-03 -5.18424722e-02
 -2.02339874e-02 -1.16729092e-02 -8.71947155e-03  1.10225017e-01
  5.62644726e-02 -8.19038999e-03  0.00000000e+00 -2.46760745e-03
 -8.08706191e-03 -5.12842356e-03 -1.23152523e-03 -1.23152523e-03
  1.00123001e+00  0.00000000e+00 -4.97218332e-03 -1.23152523e-03
 -1.74271179e-03  1.07930046e-01 -5.12842356e-03 -3.71288845e-03
 -5.12842356e-03 -5.98775832e-03]
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00]
[ 1.07014234e-02  2.65027991e-02  5.56817585e-03  1.73174278e-02
  4.10212153e-02  5.30379647e-02  4.73421107e-02 -5.66411771e-02
 -3.04951681e-02  1.20730225e-02  2.23430048e-02  4.45024573e-01
  3.61546159e-01  8.70988445e-02  0.00000000e+00 -9.96276511e-03
  8.88939494e-02  4.12589420e-02 -4.97218332e-03 -4.97218332e-03
 -4.97218332e-03  0.00000000e+00  1.00123001e+00  2.47987643e-01
 -7.03605763e-03  5.25150868e-01 -2.07055945e-02  6.97474012e-02
 -2.07055945e-02  2.92991568e-02]
[-2.80546996e-02 -1.12012685e-02  1.22033210e-02 -3.15283445e-02
  8.44457308e-02  1.01378310e-01  1.01785119e-01 -5.18424722e-02
 -2.02339874e-02 -1.16729092e-02 -8.71947155e-03  1.10225017e-01
  1.22833195e-01 -8.19038999e-03  0.00000000e+00 -2.46760745e-03
 -8.08706191e-03 -5.12842356e-03 -1.23152523e-03 -1.23152523e-03
 -1.23152523e-03  0.00000000e+00  2.47987643e-01  1.00123001e+00
 -1.74271179e-03  1.07930046e-01 -5.12842356e-03 -3.71288845e-03
 -5.12842356e-03 -5.98775832e-03]
```

```
[ 4.14078886e-02  1.50905099e-02  3.47740093e-02 -2.69857346e-02
  4.94176832e-02  5.20909535e-02 -8.34474571e-03  3.36568700e-02
  5.33990851e-02 -1.65181484e-02 -1.23387857e-02  1.55977671e-01
  7.96189615e-02 -1.15900908e-02  0.00000000e+00 -3.49187211e-03
 -1.14438729e-02 -7.25715071e-03 -1.74271179e-03 -1.74271179e-03
 -1.74271179e-03  0.00000000e+00 -7.03605763e-03 -1.74271179e-03
  1.00123001e+00  6.91801632e-02  3.40232301e-01 -5.25404946e-03
  3.40232301e-01  1.41465294e-01]
[ 6.42336539e-03  2.86925254e-02 -2.83559224e-03  3.64680887e-02
  8.96114096e-02  6.57623118e-02  2.78360150e-02 -4.44109086e-02
 -2.62997918e-02  2.81112744e-02  1.11113178e-02  9.08875351e-01
  8.95216056e-01  6.99101830e-01  0.00000000e+00  2.16259463e-01
  6.89831815e-01  4.20525796e-01  1.07930046e-01  1.07930046e-01
  1.07930046e-01  0.00000000e+00  5.25150868e-01  1.07930046e-01
  6.91801632e-02  1.00123001e+00 -1.33613063e-02  8.94156064e-03
 -1.33613063e-02  5.51994872e-04]
[ 1.03802854e-01  2.29837764e-02  5.38483722e-02  3.96053623e-02
 -1.22802468e-02  5.73406077e-02  1.11905355e-01  6.41788992e-03
  6.06757904e-02  9.50653157e-02  8.00728821e-02  1.28973977e-02
 -1.02967599e-02 -3.41071283e-02  0.00000000e+00 -1.02758237e-02
 -3.36768406e-02 -2.13562236e-02 -5.12842356e-03 -5.12842356e-03
 -5.12842356e-03  0.00000000e+00 -2.07055945e-02 -5.12842356e-03
  3.40232301e-01 -1.33613063e-02  1.00123001e+00 -1.54615302e-02
  8.80925749e-01  6.49897520e-01]
[ 6.21630731e-02  1.75015630e-02 -3.36554750e-02 -3.24023943e-03
 -4.40218269e-02 -3.17707090e-02 -2.15318281e-02 -4.29493572e-03
  2.10885381e-03  4.33990137e-02  8.15343758e-03  6.94506978e-03
 -8.76641186e-03 -2.46929609e-02  0.00000000e+00 -7.43951562e-03
 -2.43814402e-02 -1.54615302e-02 -3.71288845e-03 -3.71288845e-03
 -3.71288845e-03  0.00000000e+00  6.97474012e-02 -3.71288845e-03
 -5.25404946e-03  8.94156064e-03 -1.54615302e-02  1.00123001e+00
 -1.54615302e-02  6.20842584e-01]
[ 9.47770542e-02  2.83398583e-02  2.96063674e-02  5.18123918e-02
  1.19821826e-02  6.00124390e-02  1.13861575e-01  2.49432885e-02
  6.34448140e-02  3.75954553e-02  1.02429277e-02  1.28973977e-02
 -1.02967599e-02 -3.41071283e-02  0.00000000e+00 -1.02758237e-02
 -3.36768406e-02 -2.13562236e-02 -5.12842356e-03 -5.12842356e-03
 -5.12842356e-03  0.00000000e+00 -2.07055945e-02 -5.12842356e-03
  3.40232301e-01 -1.33613063e-02  8.80925749e-01 -1.54615302e-02
  1.00123001e+00  5.97987346e-01]
[ 8.60181742e-02  2.41160694e-02  2.27226648e-02  1.52581140e-02
 -7.09938001e-02 -5.12364780e-02 -3.47242813e-02 -1.22554484e-02
 -1.41501443e-02  1.16829514e-01  7.97764153e-02 -3.05844435e-03
 -2.19554654e-02 -3.98222258e-02  0.00000000e+00 -1.19976730e-02
 -3.93198377e-02 -2.49347395e-02 -5.98775832e-03 -5.98775832e-03
 -5.98775832e-03  0.00000000e+00  2.92991568e-02 -5.98775832e-03
  1.41465294e-01  5.51994872e-04  6.49897520e-01  6.20842584e-01
  5.97987346e-01  1.00123001e+00]]
```

4. Después de esto, siguen los eigenvalores y eigenvectores.

In [381…

```
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

print("Eigenvalues:\n", eigenvalues)
print("Eigenvectors (columns correspond to eigenvalues):\n", eigenvectors)
```

```
Eigenvalues:
 [ 4.72977368e+00  2.83757174e+00  2.45569212e+00  2.04207761e+00
  1.56169383e+00  1.45928413e+00  1.30294226e+00  1.28687883e+00
  1.15179615e+00  1.03458136e+00  1.02302484e+00  9.98147619e-01
  9.00601517e-01  8.42103993e-01  8.27483991e-01  7.43963945e-01
  7.24273248e-01  5.88485003e-01  4.79929408e-01  2.74860202e-01
  2.40838413e-01  1.74284229e-01  1.31599248e-01  8.91098674e-02
  7.43376238e-02  5.04951732e-02  8.61032352e-03  7.51245094e-17
 -1.55833146e-16 -1.55340090e-15]
Eigenvectors (columns correspond to eigenvalues):
 [[-2.03202530e-02  2.78087938e-01 -1.24184797e-01  3.80007529e-01
  -1.85475595e-01  7.82170961e-02 -2.88467332e-01  1.78329260e-01
   2.15919068e-01 -6.07994992e-02 -4.84432571e-02 -5.84052965e-02
   1.57105809e-01  1.19111870e-02  2.93519796e-02  4.17946087e-02
   1.18581536e-01 -4.46239145e-02  1.14295647e-02  6.81911659e-01
  -1.21553385e-01  1.53607536e-01  6.04176944e-02 -2.89601017e-03
  -1.06892943e-02 -7.22786876e-03  1.43648819e-03  0.00000000e+00
   0.00000000e+00  0.00000000e+00]
 [-1.99977630e-02  1.16113080e-01 -1.89152636e-01 -6.13581492e-02
   6.91182442e-02  1.16182114e-01  2.78053521e-01 -6.09290173e-02
   1.11275738e-01 -1.93254526e-01 -2.72770589e-01  6.84113853e-02
   6.12217837e-01  8.17168579e-02  4.32513344e-01  1.70874163e-01
  -3.14864341e-01  3.30269813e-02  9.61673671e-02 -7.91541245e-02
  -2.19821682e-02 -6.25262525e-02 -2.23011751e-02  6.00736291e-03
   5.84198656e-03  4.73797909e-04  7.51282441e-04  1.09480954e-18
  -7.91352161e-18  2.63041199e-16]
 [-8.12764081e-03  3.53205064e-02  8.96365000e-02  1.35545090e-01
  -2.05250921e-01 -1.66692766e-01 -5.85702455e-01  2.84816294e-01
   8.61204124e-02 -1.11988280e-01  9.63845686e-03 -4.36532811e-02
   1.87383032e-01 -1.16630250e-01 -1.76029160e-01  1.13344749e-01
  -4.08194599e-01  1.33449514e-01 -6.69408230e-02 -4.07162679e-01
   6.96148888e-02 -3.87384930e-02 -4.88280086e-02 -5.45411578e-03
   1.28774720e-02 -2.85576139e-03 -7.95405616e-03  1.14070094e-16
  -2.93559522e-16 -1.35527040e-15]
 [-2.33945440e-02  2.10700360e-01 -1.68131050e-01  3.22375653e-01
  -1.67895795e-01  2.33860310e-01 -3.18188814e-03 -2.51930755e-02
   2.67498473e-01  1.64799186e-01 -2.70163534e-03  3.55108191e-03
  -4.08309097e-02  2.24249180e-01  1.84671340e-01 -1.01409595e-01
   4.76207448e-01  2.77729581e-01 -5.04912698e-02 -4.85341852e-01
   4.48605263e-02 -3.84595747e-03  3.05412757e-03  3.00627514e-03
   1.17211434e-02 -6.89891313e-03 -4.37760379e-03 -4.26365217e-16
   4.52360323e-16 -1.28179472e-15]
 [-7.93541141e-02  1.70651871e-01 -4.42636388e-01 -2.18086133e-01
   5.32171754e-02 -5.18096716e-02  7.89098913e-02  3.44982005e-02
  -2.08303599e-03 -8.82734881e-03  5.70758889e-02 -2.15138457e-02
  -1.56041058e-02 -1.51566112e-01 -6.62801173e-02  4.40980833e-02
  -4.71099806e-02 -2.59716370e-02 -6.88455189e-01 -5.36831147e-02
   5.03229403e-02  4.30889988e-01  7.16887184e-02  5.93412179e-03
```

```
  3.91667196e-02 -1.22264880e-02 -6.15819034e-05 -1.86216138e-16
  1.10702180e-16 -1.28436508e-15]
[-7.42759710e-02  2.55669313e-01 -4.80719595e-01 -1.49616959e-01
  6.86663945e-02 -1.07338579e-01 -4.31572970e-02  9.03952497e-02
 -2.40049439e-03 -1.84001408e-02  7.67703966e-02 -3.37173819e-03
 -8.80834380e-02 -7.52607976e-02 -1.15661623e-01 -5.19919865e-02
  3.58395789e-02  2.61361678e-02 -3.01099903e-02  8.95972517e-02
 -1.89985966e-02 -7.62756090e-01 -1.40655226e-01  1.13345132e-03
 -4.94243357e-03  2.49836441e-02 -1.10785330e-02  1.75063209e-16
 -3.10503328e-16 -2.16554376e-16]
[-4.38732242e-02  2.39384692e-01 -4.13825507e-01 -1.75647726e-01
  8.03019701e-02 -9.90869613e-02 -3.55581669e-02  7.05881896e-02
 -5.50744802e-02 -3.00903205e-02  6.05441959e-02  2.14031491e-02
 -6.96123183e-02  8.32993903e-02 -2.17621465e-01 -7.97346964e-02
 -4.82089100e-02  4.46685348e-03  6.70174829e-01 -1.25100866e-01
  2.58593199e-02  3.90145840e-01  1.20348346e-01 -3.44634251e-02
 -8.06070821e-02 -1.98993414e-02  3.60859702e-03  8.53117381e-17
  2.06508605e-17  3.32881535e-16]
[ 2.13433069e-02  5.32383372e-02 -3.59966111e-02  9.94704560e-02
 -4.62468237e-01  1.09087878e-01  2.95652599e-01  3.64782925e-02
 -3.98318183e-01 -5.17003117e-02  3.91205447e-02  8.43663891e-02
 -1.56297353e-01 -2.88933431e-02 -3.61841564e-02 -5.44948923e-02
 -2.75739343e-01  6.10352966e-01 -3.70370790e-03  1.46275109e-01
 -3.33888865e-02  1.02814494e-02  7.31161777e-03  1.02710972e-02
  8.48819870e-03 -1.88874302e-03  7.25447008e-03 -2.40148262e-16
  4.19163181e-16  7.19861381e-16]
[-2.85805538e-03  1.46122449e-01 -7.88369509e-02  2.21664648e-01
 -4.62919238e-01  1.10368161e-01  1.55984924e-01  6.16774543e-02
 -2.86899172e-01  3.19597955e-02 -6.17735839e-02  1.01212531e-01
 -1.02588404e-01 -3.84922593e-02  2.68542329e-02  4.57426405e-02
 -8.05226901e-02 -7.09709548e-01  7.24995205e-03 -1.98078106e-01
 -4.62810860e-02 -5.30603427e-02  8.21211985e-03 -3.42672625e-02
 -2.80637358e-02  3.81588979e-03 -2.55599680e-03  5.32107533e-16
 -7.85913165e-16 -2.49341034e-16]
[-3.93995213e-02  1.98274742e-01  1.51621634e-02  4.83378867e-01
  3.28707722e-01 -1.04230333e-01  9.31122416e-02 -1.64695723e-01
 -1.45362765e-01 -3.76562303e-02  3.91788080e-02  1.06118074e-02
 -6.02400823e-02 -8.27202706e-02 -7.18246996e-03  3.73525331e-02
 -1.26465025e-01 -9.39087083e-03  2.96735716e-03  4.82252912e-02
  7.17026743e-01 -1.29419573e-02 -1.26375891e-02 -1.15039417e-02
  2.72687572e-02 -1.60030524e-02  2.06302772e-02 -7.97225882e-16
  1.34806266e-15  1.48763103e-15]
[-2.27617524e-02  1.69782598e-01  4.68815252e-03  4.45017365e-01
  3.98639308e-01 -1.10064510e-01  5.51995429e-02 -2.24960452e-01
 -1.32788026e-01 -4.26874384e-02  6.26867613e-02 -9.83758373e-03
 -5.76366884e-02 -1.21274367e-01 -8.14069947e-02  1.88870076e-02
 -1.33656837e-01  4.47371217e-02 -3.65999243e-02 -1.49832070e-01
 -6.71147076e-01  3.54185714e-02 -6.03936800e-02 -3.47892896e-03
```

```
 -1.37502247e-02  7.49029038e-03 -1.28365272e-02  9.24420817e-16
 -1.44800986e-15 -1.27540394e-15]
[-4.33174221e-01  1.96685721e-03  5.06172991e-02 -2.18365294e-02
 -2.52062073e-02  1.54214508e-01 -2.97518354e-02 -9.11226231e-02
  3.55835586e-02 -4.83900473e-02  4.07962411e-02 -4.80058164e-03
 -1.04436453e-02 -8.14964000e-02 -2.65680150e-02  1.15779201e-03
 -2.12286238e-02 -4.32535072e-03  2.08759830e-02 -5.22032636e-03
  2.59120654e-02  2.70691899e-02 -2.53922358e-02  3.94790798e-01
 -2.81948019e-01  7.21389300e-01  6.49604932e-02  1.63317003e-15
  3.26686031e-15  5.87499117e-14]
[-4.55557264e-01 -2.49051371e-02  5.01894682e-02 -5.18048220e-03
 -2.23837097e-02 -2.90846060e-02  1.17568033e-02  9.60585996e-03
 -1.10169582e-02  1.50805045e-02 -1.05534391e-02  2.58584259e-04
 -7.09550759e-03  8.49054274e-03  1.47119173e-02  3.61973391e-02
  1.52289099e-02  4.55063806e-03  1.43599706e-02  5.01851995e-03
 -1.25217946e-02 -4.17262631e-03 -1.62966885e-02  1.17552843e-01
 -3.57808060e-02 -3.41316660e-01 -3.78025029e-02 -3.73977438e-02
 -3.47708607e-02 -8.05641514e-01]
[-4.02029820e-01 -5.15875127e-02  6.08101854e-02  5.11176841e-02
 -5.44014601e-02 -2.83303048e-01  1.42746485e-01  1.08033974e-01
 -3.13484056e-02  3.81392743e-02 -6.63880441e-02  2.94887557e-02
  9.85120644e-02  8.90082759e-02 -8.46715013e-02  8.72663351e-02
  1.22185146e-01  2.40770002e-02 -1.05439735e-03  6.15247150e-03
 -6.31761043e-04  7.87011703e-03 -5.32485992e-03  4.18574474e-02
 -2.77752051e-03 -1.04526960e-01 -7.28749545e-01  1.56832449e-02
  1.45816263e-02  3.37856563e-01]
[ 0.00000000e+00  0.00000000e+00  2.03287907e-20  1.08420217e-19
  3.46944695e-17  2.77555756e-16  2.22044605e-16  6.93889390e-17
  3.88578059e-16  3.74700271e-16  4.99600361e-16  3.88578059e-16
  8.32667268e-17 -1.70002901e-16  1.87350135e-16  5.55111512e-17
  2.22044605e-16  7.77156117e-16  1.94289029e-16  2.42861287e-16
  1.72084569e-15  1.66533454e-16  8.88178420e-16 -4.47558657e-16
  2.22738494e-15  1.36002321e-15  4.77395901e-15  7.10847245e-03
 -9.99058859e-01  4.27886286e-02]
[-1.53120968e-01 -3.53129001e-03 -1.71459439e-02 -4.49143956e-03
  1.42065361e-02 -3.11747672e-01 -6.13294824e-02  1.99179853e-01
  1.42702614e-01  3.91958729e-02  6.65337737e-02 -3.20770681e-02
 -4.59891453e-01 -2.08449896e-02  7.08079754e-01 -1.42980488e-01
 -2.28591499e-01 -1.54743890e-02  4.26855139e-02  2.11073767e-02
 -4.36063684e-02  4.86658771e-02  7.26140930e-03  1.02454507e-02
 -4.01143652e-03 -1.25817202e-02  5.89811440e-02  4.95764169e-03
  4.60940824e-03  1.06800078e-01]
[-3.97078093e-01 -5.13711238e-02  5.95051838e-02  4.75265933e-02
 -5.84012682e-02 -2.67245074e-01  1.42602007e-01  1.00754378e-01
 -3.71753302e-02  3.60846330e-02 -7.29439917e-02  3.11280555e-02
  1.42012777e-01  9.27787437e-02 -1.50952285e-01  1.02956516e-01
  1.48553756e-01  2.09103831e-02 -1.21534176e-02 -8.96721519e-03
 -3.13654288e-02 -1.46745044e-02 -1.21270595e-02  6.57392769e-02
```

```
 -1.59851419e-02 -2.48854487e-01  6.74410499e-01  1.55054475e-02
  1.44163177e-02  3.34026359e-01]
[-1.53282303e-01 -7.98247590e-03 -2.97783532e-02 -2.94877726e-02
 -7.12098897e-02  4.09027597e-01  6.96214304e-02 -2.57083131e-01
  3.72800239e-01  1.48725357e-01  2.35284696e-01 -3.24096245e-01
 -1.97708444e-01 -4.72777266e-02 -1.14148377e-01  2.84496052e-01
 -3.32736062e-01 -4.03166865e-02  7.57100480e-02  2.18205462e-02
  9.40389646e-03 -3.69915032e-02 -3.24290284e-02  8.71880259e-02
 -1.76320889e-02 -3.11025061e-01 -3.14372764e-02  1.01380925e-02
  9.42597513e-03  2.18400026e-01]
[-2.87103356e-02 -1.23673487e-02  1.50554913e-02 -1.71093951e-02
 -1.85781402e-02  7.46553945e-02 -1.49253523e-02 -4.56382630e-02
 -2.58237198e-02 -8.53543456e-01 -2.10735776e-01 -3.11062805e-01
 -2.53336130e-01  1.03411384e-01 -1.82822499e-02  1.95826790e-03
  1.57088910e-01 -1.77297536e-02 -3.22694596e-02 -6.92631411e-02
  1.13716380e-02 -3.60049007e-03 -7.54543474e-03  3.42440112e-02
 -8.22646526e-03 -9.71459181e-02 -1.07049218e-02  2.48340701e-03
  2.30896814e-03  5.34988366e-02]
[-2.82748812e-02 -2.16306392e-02  2.70469472e-02 -2.27700332e-02
 -2.15959114e-02  5.14203597e-02 -7.55279220e-02 -4.34354014e-02
 -8.34739447e-02 -3.21356577e-01  8.07606072e-01  3.51795375e-01
  1.82617931e-01  1.22782543e-01  1.52121201e-01  5.37055672e-02
  9.77840096e-02 -5.12069320e-02 -1.33887637e-02  2.63145851e-03
 -3.03837018e-03  2.28210895e-03 -4.18745582e-03  3.43911851e-02
 -9.23055782e-03 -9.72432118e-02 -1.02830750e-02  2.48340701e-03
  2.30896814e-03  5.34988366e-02]
[-2.96024987e-02 -7.65553145e-03  7.12710879e-03 -5.57649985e-03
  3.27260797e-02  1.06175292e-01 -4.64591458e-02 -9.35270523e-02
  3.25694397e-01 -1.38031641e-01 -2.93867522e-01  7.98173130e-01
 -2.76082881e-01 -5.85037006e-02 -1.46321364e-01  3.57925678e-02
 -9.82608636e-02  1.58247272e-02 -2.41218641e-02  2.65490170e-02
 -1.04425713e-03  1.31441542e-03 -6.18109273e-03  3.43428017e-02
 -9.12386303e-03 -9.67682513e-02 -9.79603175e-03  2.48340701e-03
  2.30896814e-03  5.34988366e-02]
[ 5.70509014e-26  2.41331357e-22  1.18141166e-20  7.31686747e-20
  2.54719748e-17  2.05131051e-16  1.43982049e-16  4.51028104e-17
  2.46330734e-16  3.33066907e-16  2.39391840e-16  1.11022302e-16
 -6.93889390e-18  1.57859836e-16  1.90819582e-16  2.22044605e-16
  9.71445147e-17  5.27355937e-16  0.00000000e+00  1.70002901e-16
  1.13797860e-15  2.77555756e-17  8.04911693e-16  3.33066907e-16
  1.33226763e-15  1.55431223e-15  3.83026943e-15  9.98901042e-01
  5.11197307e-03 -4.65894474e-02]
[-1.82601727e-01  9.10820856e-03 -6.59589085e-04 -5.07895601e-02
  1.75865749e-01  4.07981820e-01 -2.92358207e-01  6.45010522e-03
 -3.12996337e-01  7.47905925e-02 -1.22231827e-01  2.35517970e-02
  1.25434391e-01 -6.44028105e-03  7.79236764e-02 -5.97569495e-01
 -8.88756600e-02 -4.01018758e-02 -5.90038206e-02  1.21190979e-02
  1.09586099e-02  2.06325855e-03 -2.09180938e-02  1.79206053e-01
```

```
 -6.65261821e-02 -2.94591686e-01 -3.09306165e-02  9.84156278e-03
  9.15027414e-03  2.12012029e-01]
[-5.49461612e-02  1.64926635e-02 -7.05816536e-02 -1.01402133e-01
  1.65881392e-01  2.03372057e-01 -3.53318020e-01  1.18721405e-02
 -4.46408622e-01  1.36434653e-01 -1.49200409e-01  7.15330682e-02
 -1.64993626e-01  1.84631062e-01  1.99081485e-01  6.43818632e-01
  1.27123086e-01  8.01424178e-02  4.47723527e-03  2.02162613e-02
 -1.47350615e-02  4.07173053e-03 -1.47229687e-03 -1.99391576e-02
  1.79965358e-02  5.32770708e-03  1.41002175e-03  2.48340701e-03
  2.30896814e-03  5.34988366e-02]
[-3.22306312e-02  2.12391797e-01  1.27363433e-01 -1.55724031e-01
 -1.57017811e-01 -5.67159196e-02 -9.03823467e-02 -2.61530514e-01
 -5.10868318e-02 -3.13754758e-02 -3.32361831e-02  4.70225605e-03
  8.79143449e-02 -7.82332396e-01  1.80987130e-01  7.64469998e-02
  2.92459039e-01  8.01563992e-02  1.44077727e-01 -4.71706856e-02
  1.69363774e-02  1.46461627e-02  5.95078309e-02 -5.24705671e-02
 -4.90101429e-02 -1.25900884e-01 -1.08115229e-02  3.50990727e-03
  3.26336522e-03  7.56122353e-02]
[-4.24994544e-01 -2.03052653e-02  4.67020043e-02 -3.02072026e-02
  1.35791661e-02  1.87086633e-01 -4.88100092e-02 -6.36596475e-02
  1.34735988e-02 -4.64262903e-02  2.88467053e-02  4.00082979e-03
  1.20703806e-03 -6.83564908e-03 -2.61079016e-02 -1.12530354e-01
 -5.56537241e-02 -1.85741193e-03  1.28439001e-02  9.51470185e-03
 -1.36268298e-02 -1.56910036e-02  8.04857550e-02 -7.40778287e-01
  3.76519145e-01  2.35040265e-01  4.05690470e-02 -8.37813754e-16
  1.64656554e-15  1.95396975e-16]
[ 7.76682387e-03  4.55516319e-01  2.63550357e-01 -1.82466595e-01
 -5.29241799e-02 -1.15886057e-01 -4.60891214e-02 -1.90236392e-01
 -4.33175903e-03  2.90251224e-02 -1.11961213e-02 -6.18614685e-03
 -1.83050035e-02  2.11088507e-01 -2.72779620e-02 -3.72236293e-02
 -6.82287350e-02 -2.73728968e-02 -2.83479238e-02 -3.61910839e-03
 -3.27785598e-02 -8.15401492e-02  4.16057455e-01  3.36181590e-01
  5.33958688e-01  4.36715258e-02  3.71381716e-03 -1.18829615e-15
  1.25382411e-15 -1.47740434e-15]
[ 5.51790782e-03  1.27347473e-01  1.57351205e-01 -1.14169179e-02
  2.20039627e-01  2.86111604e-01  2.44339763e-01  6.31287800e-01
  3.10122863e-02 -3.89785932e-02  6.54958226e-02  1.39549211e-02
 -4.36919611e-02 -2.60754670e-01 -3.79793126e-02  8.48038969e-02
  8.31564149e-02 -4.64116297e-03  9.56136101e-02 -4.96466287e-02
 -1.21144015e-02  6.98161931e-02 -2.86110493e-01  1.74041029e-01
  3.84970781e-01  3.05861876e-02  2.33064639e-03 -5.73141919e-16
  6.83008428e-16 -1.96947744e-15]
[ 8.04137543e-03  4.42011707e-01  2.47931734e-01 -2.13385234e-01
 -9.31209535e-02 -1.04771405e-01 -3.83439246e-02 -1.88372151e-01
  2.60973613e-03  3.80527070e-02 -1.95377360e-02 -2.09059092e-03
 -1.52975099e-02  2.26662992e-01 -7.76906538e-03 -5.18727228e-02
 -2.64705361e-02 -1.55043973e-02 -4.53945792e-02  4.67127656e-02
  2.08600645e-02  1.40344401e-01 -7.28802141e-01 -1.31817821e-01
```

```
 -1.04407616e-01  2.59439959e-03  9.09352731e-04  7.60385357e-16
 -8.12837044e-16  7.59445232e-16]
[ 1.45217492e-02  3.84420686e-01  3.35978102e-01 -1.17971506e-01
  1.53426580e-01  1.19330681e-01  1.44978869e-01  3.06576993e-01
  3.76251679e-02 -1.15617815e-03  3.60773621e-02 -3.25430778e-03
 -3.79409011e-02  6.77795392e-02 -3.97698559e-02  4.07445133e-02
 -4.12323858e-02  2.66651608e-02 -1.07639372e-01 -3.59701697e-02
 -9.49850548e-03 -1.02345349e-01  3.88904486e-01 -2.56070967e-01
 -5.61217658e-01 -5.22318466e-02 -7.10388821e-03  5.81414546e-16
 -7.51420207e-16  1.47900223e-15]]
```

5. Una vez contamos con estos eigenvectores y eigenvalores, podemos ordenarlos de mayor a menor, y encontrar cuantos componentes necesitamos para capturar al menos un 95% de la varianza total.

In [382…

```python
explained_var_ratio = eigenvalues / np.sum(eigenvalues)
cumulative_var = np.cumsum(explained_var_ratio)
print(cumulative_var)
```

```
[0.16871297 0.26993032 0.35752586 0.43036761 0.48607387 0.53812714
 0.58460362 0.63050712 0.67159216 0.7084961  0.74498782 0.78059215
 0.81271698 0.84275517 0.87227187 0.89880937 0.92464449 0.94563599
 0.96275527 0.97255965 0.98115045 0.98736724 0.99206144 0.99524003
 0.99789168 0.99969287 1.         1.         1.         1.        ]
```

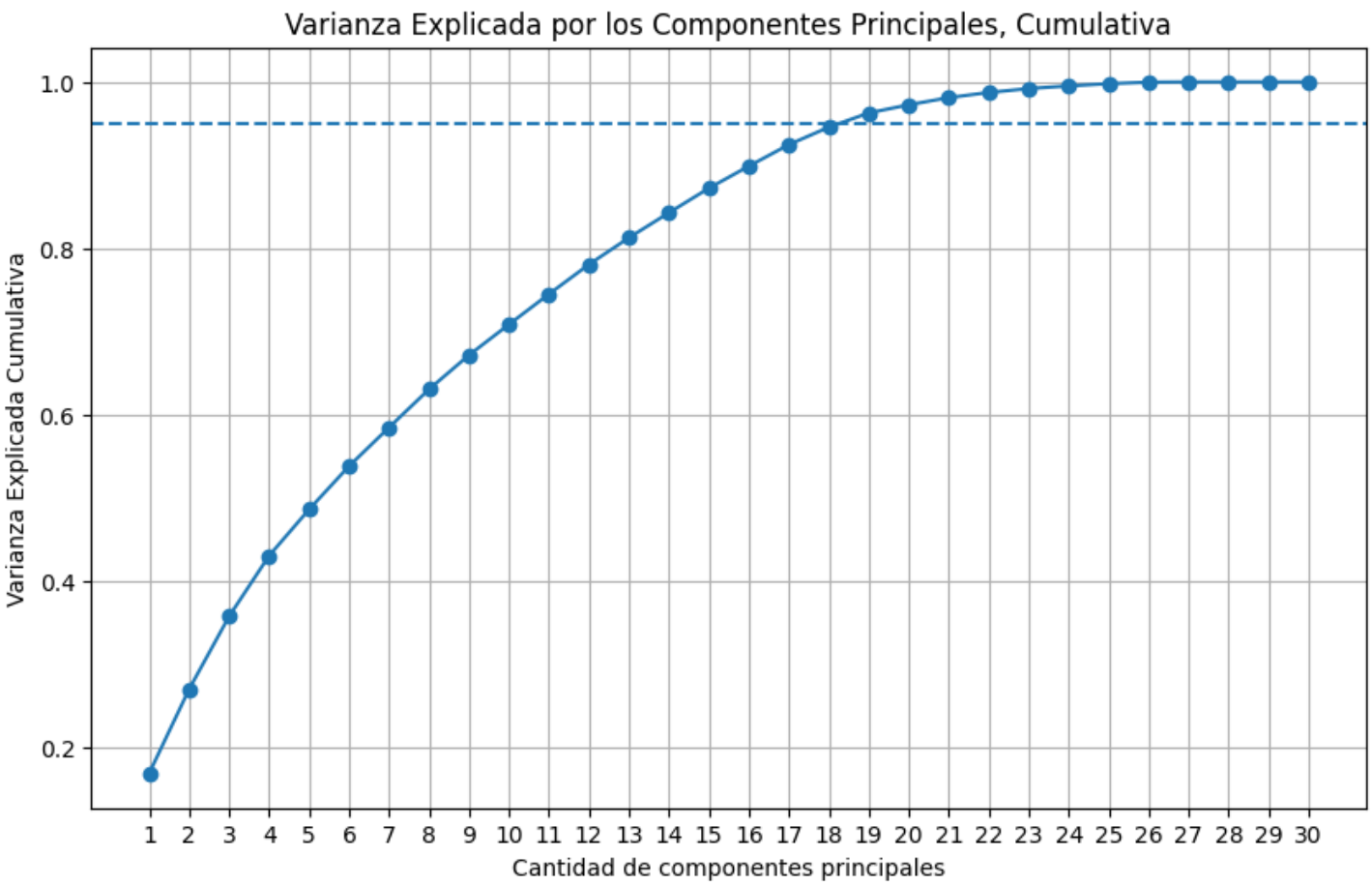
In [383…

```python
plt.figure(figsize=(10,6))
plt.plot(range(1, len(cumulative_var)+1), cumulative_var, marker='o')
plt.axhline(y=0.95, linestyle='--')
plt.grid(True)

plt.xlabel("Cantidad de componentes principales")
plt.ylabel("Varianza Explicada Cumulativa")
plt.title("Varianza Explicada por los Componentes Principales, Cumulativa")

plt.xticks(range(1, len(cumulative_var)+1)) # Línea mostrando el límite de 95%.

plt.show()
```



Como podemos observar, requeriremos 19 componentes para capturar el valor de varianza mínimo de 95%.

In [384…

```
n_components = 19
transformation_matrix = eigenvectors[:, :n_components]
transformation_matrix
```

Out[384…

```
array([[-2.03202530e-02,  2.78087938e-01, -1.24184797e-01,
         3.80007529e-01, -1.85475595e-01,  7.82170961e-02,
        -2.88467332e-01,  1.78329260e-01,  2.15919068e-01,
        -6.07994992e-02, -4.84432571e-02, -5.84052965e-02,
         1.57105809e-01,  1.19111870e-02,  2.93519796e-02,
         4.17946087e-02,  1.18581536e-01, -4.46239145e-02,
         1.14295647e-02],
       [-1.99977630e-02,  1.16113080e-01, -1.89152636e-01,
        -6.13581492e-02,  6.91182442e-02,  1.16182114e-01,
         2.78053521e-01, -6.09290173e-02,  1.11275738e-01,
        -1.93254526e-01, -2.72770589e-01,  6.84113853e-02,
         6.12217837e-01,  8.17168579e-02,  4.32513344e-01,
         1.70874163e-01, -3.14864341e-01,  3.30269813e-02,
         9.61673671e-02],
       [-8.12764081e-03,  3.53205064e-02,  8.96365000e-02,
         1.35545090e-01, -2.05250921e-01, -1.66692766e-01,
        -5.85702455e-01,  2.84816294e-01,  8.61204124e-02,
        -1.11988280e-01,  9.63845686e-03, -4.36532811e-02,
         1.87383032e-01, -1.16630250e-01, -1.76029160e-01,
         1.13344749e-01, -4.08194599e-01,  1.33449514e-01,
        -6.69408230e-02],
       [-2.33945440e-02,  2.10700360e-01, -1.68131050e-01,
         3.22375653e-01, -1.67895795e-01,  2.33860310e-01,
        -3.18188814e-03, -2.51930755e-02,  2.67498473e-01,
         1.64799186e-01, -2.70163534e-03,  3.55108191e-03,
        -4.08309097e-02,  2.24249180e-01,  1.84671340e-01,
        -1.01409595e-01,  4.76207448e-01,  2.77729581e-01,
        -5.04912698e-02],
       [-7.93541141e-02,  1.70651871e-01, -4.42636388e-01,
        -2.18086133e-01,  5.32171754e-02, -5.18096716e-02,
         7.89098913e-02,  3.44982005e-02, -2.08303599e-03,
        -8.82734881e-03,  5.70758889e-02, -2.15138457e-02,
        -1.56041058e-02, -1.51566112e-01, -6.62801173e-02,
         4.40980833e-02, -4.71099806e-02, -2.59716370e-02,
        -6.88455189e-01],
       [-7.42759710e-02,  2.55669313e-01, -4.80719595e-01,
        -1.49616959e-01,  6.86663945e-02, -1.07338579e-01,
        -4.31572970e-02,  9.03952497e-02, -2.40049439e-03,
        -1.84001408e-02,  7.67703966e-02, -3.37173819e-03,
        -8.80834380e-02, -7.52607976e-02, -1.15661623e-01,
        -5.19919865e-02,  3.58395789e-02,  2.61361678e-02,
        -3.01099903e-02],
       [-4.38732242e-02,  2.39384692e-01, -4.13825507e-01,
        -1.75647726e-01,  8.03019701e-02, -9.90869613e-02,
        -3.55581669e-02,  7.05881896e-02, -5.50744802e-02,
        -3.00903205e-02,  6.05441959e-02,  2.14031491e-02,
        -6.96123183e-02,  8.32993903e-02, -2.17621465e-01,
        -7.97346964e-02, -4.82089100e-02,  4.46685348e-03,
```

```
  6.70174829e-01],
[ 2.13433069e-02,  5.32383372e-02, -3.59966111e-02,
  9.94704560e-02, -4.62468237e-01,  1.09087878e-01,
  2.95652599e-01,  3.64782925e-02, -3.98318183e-01,
 -5.17003117e-02,  3.91205447e-02,  8.43663891e-02,
 -1.56297353e-01, -2.88933431e-02, -3.61841564e-02,
 -5.44948923e-02, -2.75739343e-01,  6.10352966e-01,
 -3.70370790e-03],
[-2.85805538e-03,  1.46122449e-01, -7.88369509e-02,
  2.21664648e-01, -4.62919238e-01,  1.10368161e-01,
  1.55984924e-01,  6.16774543e-02, -2.86899172e-01,
  3.19597955e-02, -6.17735839e-02,  1.01212531e-01,
 -1.02588404e-01, -3.84922593e-02,  2.68542329e-02,
  4.57426405e-02, -8.05226901e-02, -7.09709548e-01,
  7.24995205e-03],
[-3.93995213e-02,  1.98274742e-01,  1.51621634e-02,
  4.83378867e-01,  3.28707722e-01, -1.04230333e-01,
  9.31122416e-02, -1.64695723e-01, -1.45362765e-01,
 -3.76562303e-02,  3.91788080e-02,  1.06118074e-02,
 -6.02400823e-02, -8.27202706e-02, -7.18246996e-03,
  3.73525331e-02, -1.26465025e-01, -9.39087083e-03,
  2.96735716e-03],
[-2.27617524e-02,  1.69782598e-01,  4.68815252e-03,
  4.45017365e-01,  3.98639308e-01, -1.10064510e-01,
  5.51995429e-02, -2.24960452e-01, -1.32788026e-01,
 -4.26874384e-02,  6.26867613e-02, -9.83758373e-03,
 -5.76366884e-02, -1.21274367e-01, -8.14069947e-02,
  1.88870076e-02, -1.33656837e-01,  4.47371217e-02,
 -3.65999243e-02],
[-4.33174221e-01,  1.96685721e-03,  5.06172991e-02,
 -2.18365294e-02, -2.52062073e-02,  1.54214508e-01,
 -2.97518354e-02, -9.11226231e-02,  3.55835586e-02,
 -4.83900473e-02,  4.07962411e-02, -4.80058164e-03,
 -1.04436453e-02, -8.14964000e-02, -2.65680150e-02,
  1.15779201e-03, -2.12286238e-02, -4.32535072e-03,
  2.08759830e-02],
[-4.55557264e-01, -2.49051371e-02,  5.01894682e-02,
 -5.18048220e-03, -2.23837097e-02, -2.90846060e-02,
  1.17568033e-02,  9.60585996e-03, -1.10169582e-02,
  1.50805045e-02, -1.05534391e-02,  2.58584259e-04,
 -7.09550759e-03,  8.49054274e-03,  1.47119173e-02,
  3.61973391e-02,  1.52289099e-02,  4.55063806e-03,
  1.43599706e-02],
[-4.02029820e-01, -5.15875127e-02,  6.08101854e-02,
  5.11176841e-02, -5.44014601e-02, -2.83303048e-01,
  1.42746485e-01,  1.08033974e-01, -3.13484056e-02,
  3.81392743e-02, -6.63880441e-02,  2.94887557e-02,
  9.85120644e-02,  8.90082759e-02, -8.46715013e-02,
```

```
  8.72663351e-02,  1.22185146e-01,  2.40770002e-02,
 -1.05439735e-03],
[ 0.00000000e+00,  0.00000000e+00,  2.03287907e-20,
  1.08420217e-19,  3.46944695e-17,  2.77555756e-16,
  2.22044605e-16,  6.93889390e-17,  3.88578059e-16,
  3.74700271e-16,  4.99600361e-16,  3.88578059e-16,
  8.32667268e-17, -1.70002901e-16,  1.87350135e-16,
  5.55111512e-17,  2.22044605e-16,  7.77156117e-16,
  1.94289029e-16],
[-1.53120968e-01, -3.53129001e-03, -1.71459439e-02,
 -4.49143956e-03,  1.42065361e-02, -3.11747672e-01,
 -6.13294824e-02,  1.99179853e-01,  1.42702614e-01,
  3.91958729e-02,  6.65337737e-02, -3.20770681e-02,
 -4.59891453e-01, -2.08449896e-02,  7.08079754e-01,
 -1.42980488e-01, -2.28591499e-01, -1.54743890e-02,
  4.26855139e-02],
[-3.97078093e-01, -5.13711238e-02,  5.95051838e-02,
  4.75265933e-02, -5.84012682e-02, -2.67245074e-01,
  1.42602007e-01,  1.00754378e-01, -3.71753302e-02,
  3.60846330e-02, -7.29439917e-02,  3.11280555e-02,
  1.42012777e-01,  9.27787437e-02, -1.50952285e-01,
  1.02956516e-01,  1.48553756e-01,  2.09103831e-02,
 -1.21534176e-02],
[-1.53282303e-01, -7.98247590e-03, -2.97783532e-02,
 -2.94877726e-02, -7.12098897e-02,  4.09027597e-01,
  6.96214304e-02, -2.57083131e-01,  3.72800239e-01,
  1.48725357e-01,  2.35284696e-01, -3.24096245e-01,
 -1.97708444e-01, -4.72777266e-02, -1.14148377e-01,
  2.84496052e-01, -3.32736062e-01, -4.03166865e-02,
  7.57100480e-02],
[-2.87103356e-02, -1.23673487e-02,  1.50554913e-02,
 -1.71093951e-02, -1.85781402e-02,  7.46553945e-02,
 -1.49253523e-02, -4.56382630e-02, -2.58237198e-02,
 -8.53543456e-01, -2.10735776e-01, -3.11062805e-01,
 -2.53336130e-01,  1.03411384e-01, -1.82822499e-02,
  1.95826790e-03,  1.57088910e-01, -1.77297536e-02,
 -3.22694596e-02],
[-2.82748812e-02, -2.16306392e-02,  2.70469472e-02,
 -2.27700332e-02, -2.15959114e-02,  5.14203597e-02,
 -7.55279220e-02, -4.34354014e-02, -8.34739447e-02,
 -3.21356577e-01,  8.07606072e-01,  3.51795375e-01,
  1.82617931e-01,  1.22782543e-01,  1.52121201e-01,
  5.37055672e-02,  9.77840096e-02, -5.12069320e-02,
 -1.33887637e-02],
[-2.96024987e-02, -7.65553145e-03,  7.12710879e-03,
 -5.57649985e-03,  3.27260797e-02,  1.06175292e-01,
 -4.64591458e-02, -9.35270523e-02,  3.25694397e-01,
 -1.38031641e-01, -2.93867522e-01,  7.98173130e-01,
```

```
 -2.76082881e-01, -5.85037006e-02, -1.46321364e-01,
  3.57925678e-02, -9.82608636e-02,  1.58247272e-02,
 -2.41218641e-02],
[ 5.70509014e-26,  2.41331357e-22,  1.18141166e-20,
  7.31686747e-20,  2.54719748e-17,  2.05131051e-16,
  1.43982049e-16,  4.51028104e-17,  2.46330734e-16,
  3.33066907e-16,  2.39391840e-16,  1.11022302e-16,
 -6.93889390e-18,  1.57859836e-16,  1.90819582e-16,
  2.22044605e-16,  9.71445147e-17,  5.27355937e-16,
  0.00000000e+00],
[-1.82601727e-01,  9.10820856e-03, -6.59589085e-04,
 -5.07895601e-02,  1.75865749e-01,  4.07981820e-01,
 -2.92358207e-01,  6.45010522e-03, -3.12996337e-01,
  7.47905925e-02, -1.22231827e-01,  2.35517970e-02,
  1.25434391e-01, -6.44028105e-03,  7.79236764e-02,
 -5.97569495e-01, -8.88756600e-02, -4.01018758e-02,
 -5.90038206e-02],
[-5.49461612e-02,  1.64926635e-02, -7.05816536e-02,
 -1.01402133e-01,  1.65881392e-01,  2.03372057e-01,
 -3.53318020e-01,  1.18721405e-02, -4.46408622e-01,
  1.36434653e-01, -1.49200409e-01,  7.15330682e-02,
 -1.64993626e-01,  1.84631062e-01,  1.99081485e-01,
  6.43818632e-01,  1.27123086e-01,  8.01424178e-02,
  4.47723527e-03],
[-3.22306312e-02,  2.12391797e-01,  1.27363433e-01,
 -1.55724031e-01, -1.57017811e-01, -5.67159196e-02,
 -9.03823467e-02, -2.61530514e-01, -5.10868318e-02,
 -3.13754758e-02, -3.32361831e-02,  4.70225605e-03,
  8.79143449e-02, -7.82332396e-01,  1.80987130e-01,
  7.64469998e-02,  2.92459039e-01,  8.01563992e-02,
  1.44077727e-01],
[-4.24994544e-01, -2.03052653e-02,  4.67020043e-02,
 -3.02072026e-02,  1.35791661e-02,  1.87086633e-01,
 -4.88100092e-02, -6.36596475e-02,  1.34735988e-02,
 -4.64262903e-02,  2.88467053e-02,  4.00082979e-03,
  1.20703806e-03, -6.83564908e-03, -2.61079016e-02,
 -1.12530354e-01, -5.56537241e-02, -1.85741193e-03,
  1.28439001e-02],
[ 7.76682387e-03,  4.55516319e-01,  2.63550357e-01,
 -1.82466595e-01, -5.29241799e-02, -1.15886057e-01,
 -4.60891214e-02, -1.90236392e-01, -4.33175903e-03,
  2.90251224e-02, -1.11961213e-02, -6.18614685e-03,
 -1.83050035e-02,  2.11088507e-01, -2.72779620e-02,
 -3.72236293e-02, -6.82287350e-02, -2.73728968e-02,
 -2.83479238e-02],
[ 5.51790782e-03,  1.27347473e-01,  1.57351205e-01,
 -1.14169179e-02,  2.20039627e-01,  2.86111604e-01,
  2.44339763e-01,  6.31287800e-01,  3.10122863e-02,
```

```
 -3.89785932e-02,  6.54958226e-02,  1.39549211e-02,
 -4.36919611e-02, -2.60754670e-01, -3.79793126e-02,
  8.48038969e-02,  8.31564149e-02, -4.64116297e-03,
  9.56136101e-02],
[ 8.04137543e-03,  4.42011707e-01,  2.47931734e-01,
 -2.13385234e-01, -9.31209535e-02, -1.04771405e-01,
 -3.83439246e-02, -1.88372151e-01,  2.60973613e-03,
  3.80527070e-02, -1.95377360e-02, -2.09059092e-03,
 -1.52975099e-02,  2.26662992e-01, -7.76906538e-03,
 -5.18727228e-02, -2.64705361e-02, -1.55043973e-02,
 -4.53945792e-02],
[ 1.45217492e-02,  3.84420686e-01,  3.35978102e-01,
 -1.17971506e-01,  1.53426580e-01,  1.19330681e-01,
  1.44978869e-01,  3.06576993e-01,  3.76251679e-02,
 -1.15617815e-03,  3.60773621e-02, -3.25430778e-03,
 -3.79409011e-02,  6.77795392e-02, -3.97698559e-02,
  4.07445133e-02, -4.12323858e-02,  2.66651608e-02,
 -1.07639372e-01]])
```

6. Con esto, tenemos nuestra matriz de transformación, y podemos derivar nuestros componentes principales de la siguiente manera:

In [385…

```python
PC_19 = df_scaled.dot(transformation_matrix)
PC_19
```

Out[385…

```
array([[ 0.72231531, -1.04415446,  0.32438342, ...,  0.01578852,
        -0.76538767,  0.33542765],
       [ 0.76964283, -1.37076955,  0.68958816, ...,  0.70629627,
        -0.85848823,  0.17536705],
       [-0.88616408, 13.82890882, -9.57225729, ..., -0.76109389,
         0.38243822,  8.56533249],
       ...,
       [ 0.7874139 , -0.96854682,  0.56179252, ..., -0.71645746,
         0.35291918,  0.20583752],
       [ 0.71521733, -0.32305022,  0.42840374, ..., -0.93665708,
         1.03452764, -0.0202341 ],
       [ 0.75241697, -0.63513402,  0.47019873, ..., -0.76396141,
         0.58820258,  0.10543076]])
```

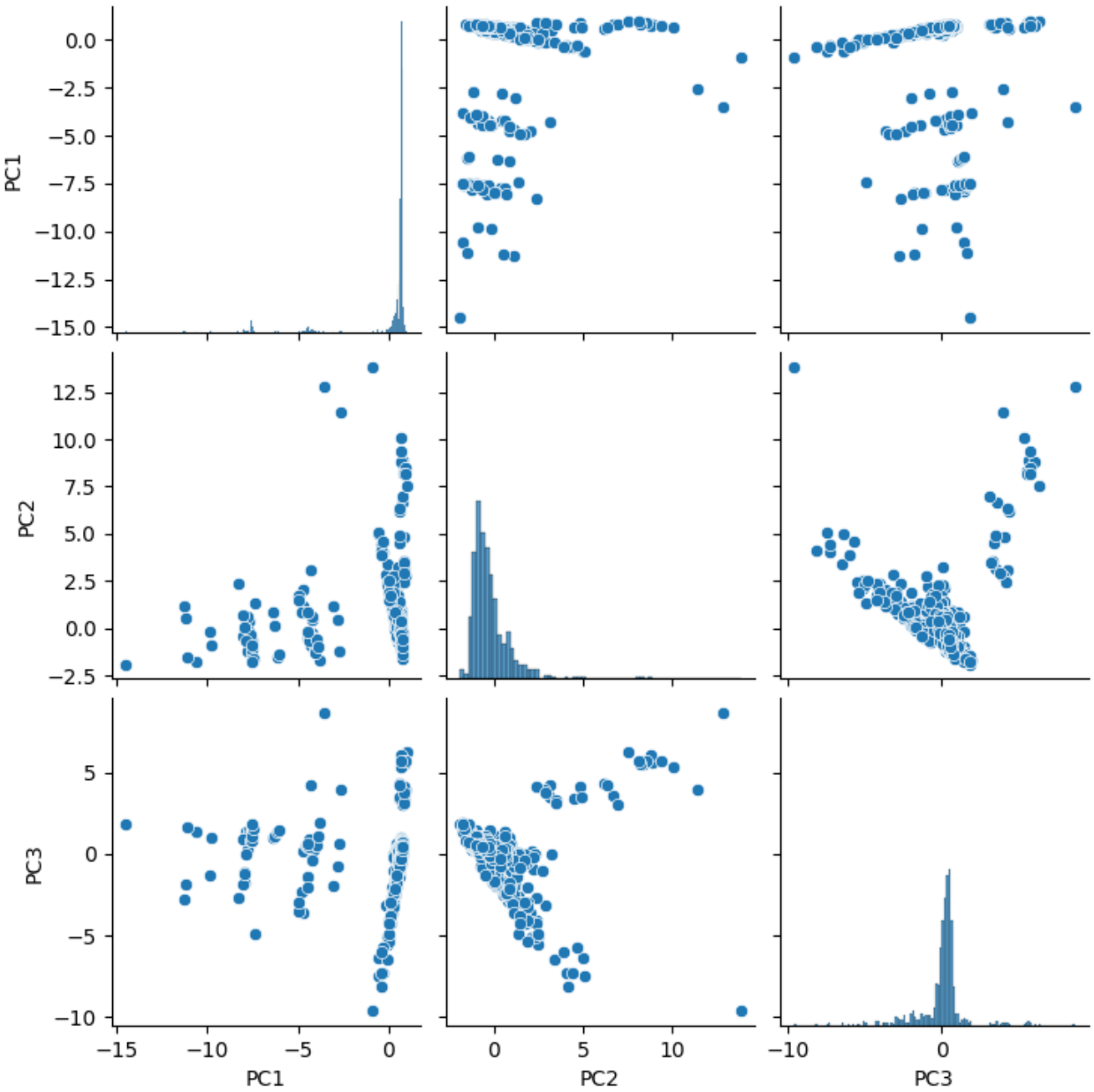
In [386…

```python
# Crear dataframe con los 19 componentes seleccionados.
df_pcs = pd.DataFrame(PC_19, columns=[f'PC{i}' for i in range(1, 20)])
```

In [387…

```
# Nueva DataFrame con los 3 primeros componentes, para visualizarlos.
pcs_subset = df_pcs[['PC1', 'PC2', 'PC3']]

# Gráficos comparando componentes en pares.
sns.pairplot(pcs_subset)
plt.show()
```



In [388…

```
from mpl_toolkits.mplot3d import Axes3D

# Crear figura 3d.
```

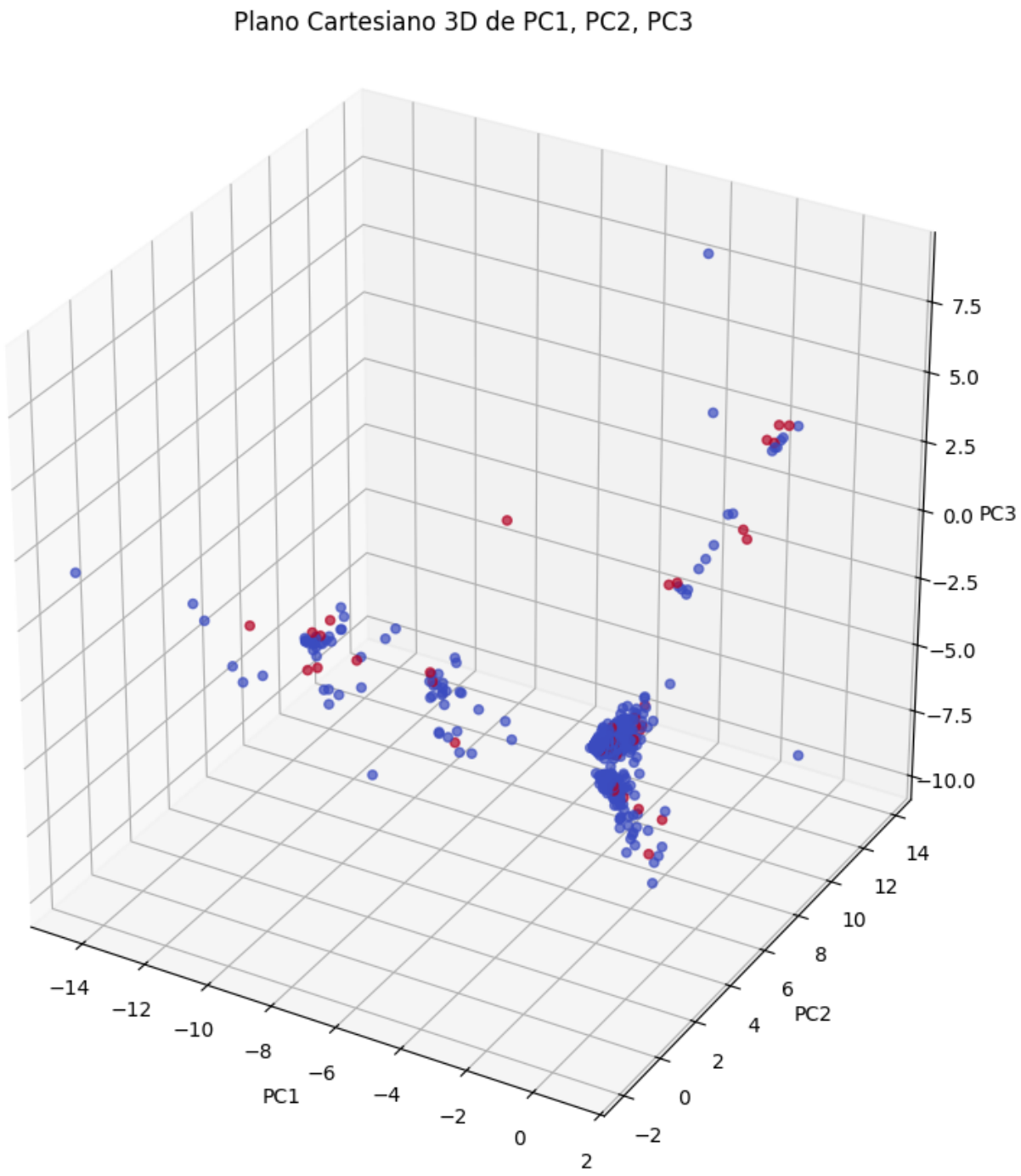
```python
fig = plt.figure(figsize=(12,10))
ax = fig.add_subplot(111, projection='3d')

# Definir escalas iguales para todos los lados.
ax.set_box_aspect([1, 1, 1])

# Visualizar datos con color basado en clase en Biopsy.
sc = ax.scatter(df_pcs['PC1'], df_pcs['PC2'], df_pcs['PC3'],
                c=df['Biopsy'], cmap='coolwarm', alpha=0.7)

ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')
ax.set_title('Plano Cartesiano 3D de PC1, PC2, PC3')


plt.show()
```


Plano Cartesiano 3D de PC1, PC2, PC3
PC1
PC2
PC3
−14
−12
−10
−8
−6
−4
−2
0
2
−2
0
2
4
6
8
10
12
14
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0

Podemos obserar una representación 3D de los 3 primeros componentes principales que hemos generado. Podemos observar como se generan agrupaciones de datos específicas, separadas entre si dependiendo de su valor en las 3 escalas, PC 1, 2, y 3.

La dimensionalidad de nuestros datos terminó siendo reducida desde 36 a tan solo 19, una reducción significativa. Como se mencionó en el análisis de la matriz de correlación, existía una cantidad significativa de redundancia en los datos, con varias variables representando datos muy similares. Esto es evidenciado debido al hecho de que para capturar casi toda la varianza en el dataset, se requiere una cantidad mínima de componentes, 19, en comparación a las 36 variables iniciales.

Modelo de una neurona: PCA

1. Se inicia separando las variables predictoras de la variable objetivo. La variable "y" sigue siendo la columna Biopsy original, y la X es nuestros componentes.

In [389…

```
y2 = df["Biopsy"]
X2 = df_pcs
```

2. Después de esto se separa el 20% de los datos para la prueba y se utiliza stratify=y para mantener la proporción de clases en cada subconjunto, porque el dataset muestra desbalance. Luego se divide el conjunto de entrenamiento.

In [390…

```
X_train_full, X_test, y_train_full, y_test = train_test_split(X2, y2, test_size=0.2, random_state=42, stratify=y)

X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full, test_size=0.2, random_state=42, stratify=y_train_full)

print("Training size:", X_train.shape)
print("Validation size:", X_val.shape)
print("Testing size:", X_test.shape)
```

```
Training size: (520, 19)
Validation size: (131, 19)
Testing size: (163, 19)
```

3. No es necesario aplicar StandardScaler de nuevo, ya que los componentes principales ya vienen en un formato estadarizado. El siguiente paso es realizar el cálculo de pesos por desbalance.

In [391…

```
class_weights = class_weight.compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)

# Transformar lista a diccionario.
```

```
class_weights = dict(enumerate(class_weights))

print("Class Weights:", class_weights)
```

```
Class Weights: {0: np.float64(0.5349794238683128), 1: np.float64(7.647058823529412)}
```

Como se hizo anteriormente, una vez más se muestra el desbalance de las clases.

4. Se define el modelo de una sola neurona y una sola capa.

In [392…

```
# Definir red neuronal, con semilla.
modelo = tf.keras.Sequential([
    tf.keras.Input(shape=(X_train.shape[1],)), # Definir shape de la entrada.
    tf.keras.layers.Dense(                     # Definir un modelo de 1 neurona.
        1,
        activation='sigmoid',
        kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)
    )
])

# Configurar entrenamiento.
modelo.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Mostrar arquitectura.
modelo.summary()
```

Model: "sequential_17"

Layer (type)	Output Shape	Param #
dense_17 (Dense)	(None, 1)	20

Total params: 20 (80.00 B)

Trainable params: 20 (80.00 B)

Non-trainable params: 0 (0.00 B)

Nuestro modelo cuenta con 20 parámetros. Esto se debe a que se tienen 19 features (componentes principales), y la neurona de output con 1 bias. Esto suma a 20.

5. Se realiza el entrenamiento de la nuerona usando los datos de entrenamiento. Se tienen 100 épocas en las cuales se ajustan los 20 parámetros. Se usa el class weight para penalizar el modelo cuando este se equivoca en la clase positiva, siendo que es la minoría.

In [393…

```
history2 = modelo.fit(
    X_train,
    y_train,
    epochs=100, # Definir hiperparámetros básicos.
    batch_size=32,
    validation_data=(X_val, y_val),
    class_weight=class_weights,
    verbose=1, # Mostrar detalle.
)
```

```
Epoch 1/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.4986 - loss: 0.8861 - val_accuracy: 0.5191 - val_loss: 0.7625
Epoch 2/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4848 - loss: 0.9159 - val_accuracy: 0.5420 - val_loss: 0.7606
Epoch 3/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4843 - loss: 0.7914 - val_accuracy: 0.5496 - val_loss: 0.7603
Epoch 4/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5405 - loss: 0.6747 - val_accuracy: 0.5573 - val_loss: 0.7587
Epoch 5/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5197 - loss: 0.8601 - val_accuracy: 0.5725 - val_loss: 0.7574
Epoch 6/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4979 - loss: 0.7080 - val_accuracy: 0.5649 - val_loss: 0.7554
Epoch 7/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4875 - loss: 0.8007 - val_accuracy: 0.5649 - val_loss: 0.7549
Epoch 8/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5164 - loss: 0.9212 - val_accuracy: 0.5573 - val_loss: 0.7533
Epoch 9/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.5652 - loss: 0.6920 - val_accuracy: 0.5573 - val_loss: 0.7524
Epoch 10/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5238 - loss: 0.7531 - val_accuracy: 0.5725 - val_loss: 0.7525
Epoch 11/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5450 - loss: 0.7838 - val_accuracy: 0.5878 - val_loss: 0.7506
Epoch 12/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.5441 - loss: 0.7010 - val_accuracy: 0.5878 - val_loss: 0.7481
Epoch 13/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5616 - loss: 0.6699 - val_accuracy: 0.5954 - val_loss: 0.7484
Epoch 14/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5534 - loss: 0.6092 - val_accuracy: 0.5954 - val_loss: 0.7467
Epoch 15/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.5509 - loss: 0.8006 - val_accuracy: 0.6031 - val_loss: 0.7460
Epoch 16/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5725 - loss: 0.7423 - val_accuracy: 0.6031 - val_loss: 0.7443
Epoch 17/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6086 - loss: 0.7863 - val_accuracy: 0.6260 - val_loss: 0.7434
Epoch 18/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5869 - loss: 0.6029 - val_accuracy: 0.6336 - val_loss: 0.7409
Epoch 19/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6232 - loss: 0.8464 - val_accuracy: 0.6336 - val_loss: 0.7428
Epoch 20/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6094 - loss: 0.6778 - val_accuracy: 0.6412 - val_loss: 0.7418
Epoch 21/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6245 - loss: 0.6709 - val_accuracy: 0.6412 - val_loss: 0.7398
Epoch 22/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6635 - loss: 0.7912 - val_accuracy: 0.6489 - val_loss: 0.7383
Epoch 23/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6387 - loss: 0.6692 - val_accuracy: 0.6641 - val_loss: 0.7364
Epoch 24/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6173 - loss: 0.7829 - val_accuracy: 0.6565 - val_loss: 0.7356
```

```
Epoch 25/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6796 - loss: 0.6837 - val_accuracy: 0.6565 - val_loss: 0.7346
Epoch 26/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6836 - loss: 0.7547 - val_accuracy: 0.6565 - val_loss: 0.7328
Epoch 27/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6852 - loss: 0.6379 - val_accuracy: 0.6565 - val_loss: 0.7314
Epoch 28/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7038 - loss: 0.6660 - val_accuracy: 0.6641 - val_loss: 0.7298
Epoch 29/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6822 - loss: 0.5845 - val_accuracy: 0.6794 - val_loss: 0.7276
Epoch 30/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7104 - loss: 0.6274 - val_accuracy: 0.6794 - val_loss: 0.7261
Epoch 31/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6936 - loss: 0.7356 - val_accuracy: 0.6870 - val_loss: 0.7265
Epoch 32/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7310 - loss: 0.5918 - val_accuracy: 0.6947 - val_loss: 0.7245
Epoch 33/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6982 - loss: 0.6593 - val_accuracy: 0.6947 - val_loss: 0.7236
Epoch 34/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7234 - loss: 0.6172 - val_accuracy: 0.6947 - val_loss: 0.7215
Epoch 35/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.6792 - loss: 0.6840 - val_accuracy: 0.6947 - val_loss: 0.7199
Epoch 36/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7161 - loss: 0.6462 - val_accuracy: 0.6947 - val_loss: 0.7202
Epoch 37/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6935 - loss: 0.5963 - val_accuracy: 0.6870 - val_loss: 0.7172
Epoch 38/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7270 - loss: 0.6440 - val_accuracy: 0.6870 - val_loss: 0.7161
Epoch 39/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7039 - loss: 0.7179 - val_accuracy: 0.6870 - val_loss: 0.7150
Epoch 40/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7263 - loss: 0.6107 - val_accuracy: 0.7023 - val_loss: 0.7126
Epoch 41/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7340 - loss: 0.6329 - val_accuracy: 0.7023 - val_loss: 0.7136
Epoch 42/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7284 - loss: 0.6803 - val_accuracy: 0.6870 - val_loss: 0.7134
Epoch 43/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7305 - loss: 0.6489 - val_accuracy: 0.7023 - val_loss: 0.7132
Epoch 44/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6933 - loss: 0.7834 - val_accuracy: 0.6870 - val_loss: 0.7124
Epoch 45/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7667 - loss: 0.6024 - val_accuracy: 0.7099 - val_loss: 0.7113
Epoch 46/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7795 - loss: 0.5709 - val_accuracy: 0.6947 - val_loss: 0.7104
Epoch 47/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7569 - loss: 0.6395 - val_accuracy: 0.7023 - val_loss: 0.7092
Epoch 48/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7440 - loss: 0.6806 - val_accuracy: 0.7023 - val_loss: 0.7079
```

```
Epoch 49/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7836 - loss: 0.6062 - val_accuracy: 0.7023 - val_loss: 0.7071
Epoch 50/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7350 - loss: 0.6323 - val_accuracy: 0.7023 - val_loss: 0.7064
Epoch 51/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7600 - loss: 0.5803 - val_accuracy: 0.7099 - val_loss: 0.7051
Epoch 52/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7526 - loss: 0.5866 - val_accuracy: 0.7099 - val_loss: 0.7033
Epoch 53/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7307 - loss: 0.7112 - val_accuracy: 0.7099 - val_loss: 0.7038
Epoch 54/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7734 - loss: 0.5878 - val_accuracy: 0.7099 - val_loss: 0.7041
Epoch 55/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7649 - loss: 0.6015 - val_accuracy: 0.7099 - val_loss: 0.7028
Epoch 56/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7614 - loss: 0.5574 - val_accuracy: 0.7099 - val_loss: 0.7016
Epoch 57/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7990 - loss: 0.5829 - val_accuracy: 0.7023 - val_loss: 0.7037
Epoch 58/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7653 - loss: 0.5692 - val_accuracy: 0.7023 - val_loss: 0.7019
Epoch 59/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7742 - loss: 0.6384 - val_accuracy: 0.7099 - val_loss: 0.7010
Epoch 60/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7756 - loss: 0.6718 - val_accuracy: 0.7099 - val_loss: 0.7004
Epoch 61/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7871 - loss: 0.5959 - val_accuracy: 0.7099 - val_loss: 0.6993
Epoch 62/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7876 - loss: 0.5816 - val_accuracy: 0.7099 - val_loss: 0.6992
Epoch 63/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.7740 - loss: 0.6416 - val_accuracy: 0.7099 - val_loss: 0.6979
Epoch 64/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7812 - loss: 0.6646 - val_accuracy: 0.7099 - val_loss: 0.6978
Epoch 65/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.7919 - loss: 0.5483 - val_accuracy: 0.7099 - val_loss: 0.6961
Epoch 66/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7914 - loss: 0.6150 - val_accuracy: 0.7176 - val_loss: 0.6959
Epoch 67/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.7831 - loss: 0.6251 - val_accuracy: 0.7176 - val_loss: 0.6955
Epoch 68/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7934 - loss: 0.6373 - val_accuracy: 0.7176 - val_loss: 0.6940
Epoch 69/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7876 - loss: 0.6176 - val_accuracy: 0.7176 - val_loss: 0.6929
Epoch 70/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8049 - loss: 0.5660 - val_accuracy: 0.7176 - val_loss: 0.6920
Epoch 71/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8089 - loss: 0.5651 - val_accuracy: 0.7176 - val_loss: 0.6929
Epoch 72/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7622 - loss: 0.6568 - val_accuracy: 0.7176 - val_loss: 0.6929
```

```
Epoch 73/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7919 - loss: 0.6470 - val_accuracy: 0.7176 - val_loss: 0.6917
Epoch 74/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8150 - loss: 0.5481 - val_accuracy: 0.7176 - val_loss: 0.6909
Epoch 75/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7866 - loss: 0.6281 - val_accuracy: 0.7176 - val_loss: 0.6910
Epoch 76/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7924 - loss: 0.6560 - val_accuracy: 0.7176 - val_loss: 0.6900
Epoch 77/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8112 - loss: 0.5156 - val_accuracy: 0.7176 - val_loss: 0.6897
Epoch 78/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7911 - loss: 0.5898 - val_accuracy: 0.7252 - val_loss: 0.6891
Epoch 79/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8062 - loss: 0.6714 - val_accuracy: 0.7252 - val_loss: 0.6901
Epoch 80/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8127 - loss: 0.6317 - val_accuracy: 0.7252 - val_loss: 0.6910
Epoch 81/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8101 - loss: 0.6245 - val_accuracy: 0.7252 - val_loss: 0.6902
Epoch 82/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8002 - loss: 0.6222 - val_accuracy: 0.7252 - val_loss: 0.6901
Epoch 83/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8075 - loss: 0.6017 - val_accuracy: 0.7176 - val_loss: 0.6894
Epoch 84/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7916 - loss: 0.5738 - val_accuracy: 0.7252 - val_loss: 0.6888
Epoch 85/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7836 - loss: 0.6158 - val_accuracy: 0.7252 - val_loss: 0.6911
Epoch 86/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7954 - loss: 0.6699 - val_accuracy: 0.7252 - val_loss: 0.6907
Epoch 87/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7983 - loss: 0.5657 - val_accuracy: 0.7252 - val_loss: 0.6895
Epoch 88/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7851 - loss: 0.6314 - val_accuracy: 0.7252 - val_loss: 0.6888
Epoch 89/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7766 - loss: 0.6787 - val_accuracy: 0.7252 - val_loss: 0.6869
Epoch 90/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8007 - loss: 0.5822 - val_accuracy: 0.7252 - val_loss: 0.6863
Epoch 91/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7957 - loss: 0.6309 - val_accuracy: 0.7252 - val_loss: 0.6851
Epoch 92/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8189 - loss: 0.5671 - val_accuracy: 0.7252 - val_loss: 0.6842
Epoch 93/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8312 - loss: 0.5893 - val_accuracy: 0.7252 - val_loss: 0.6849
Epoch 94/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7883 - loss: 0.6000 - val_accuracy: 0.7252 - val_loss: 0.6868
Epoch 95/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8181 - loss: 0.5903 - val_accuracy: 0.7252 - val_loss: 0.6865
Epoch 96/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8087 - loss: 0.5644 - val_accuracy: 0.7252 - val_loss: 0.6854
```

```
Epoch 97/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7847 - loss: 0.6378 - val_accuracy: 0.7252 - val_loss: 0.6879
Epoch 98/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8077 - loss: 0.5356 - val_accuracy: 0.7252 - val_loss: 0.6870
Epoch 99/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7965 - loss: 0.6581 - val_accuracy: 0.7252 - val_loss: 0.6884
Epoch 100/100
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7941 - loss: 0.6658 - val_accuracy: 0.7252 - val_loss: 0.6887
```

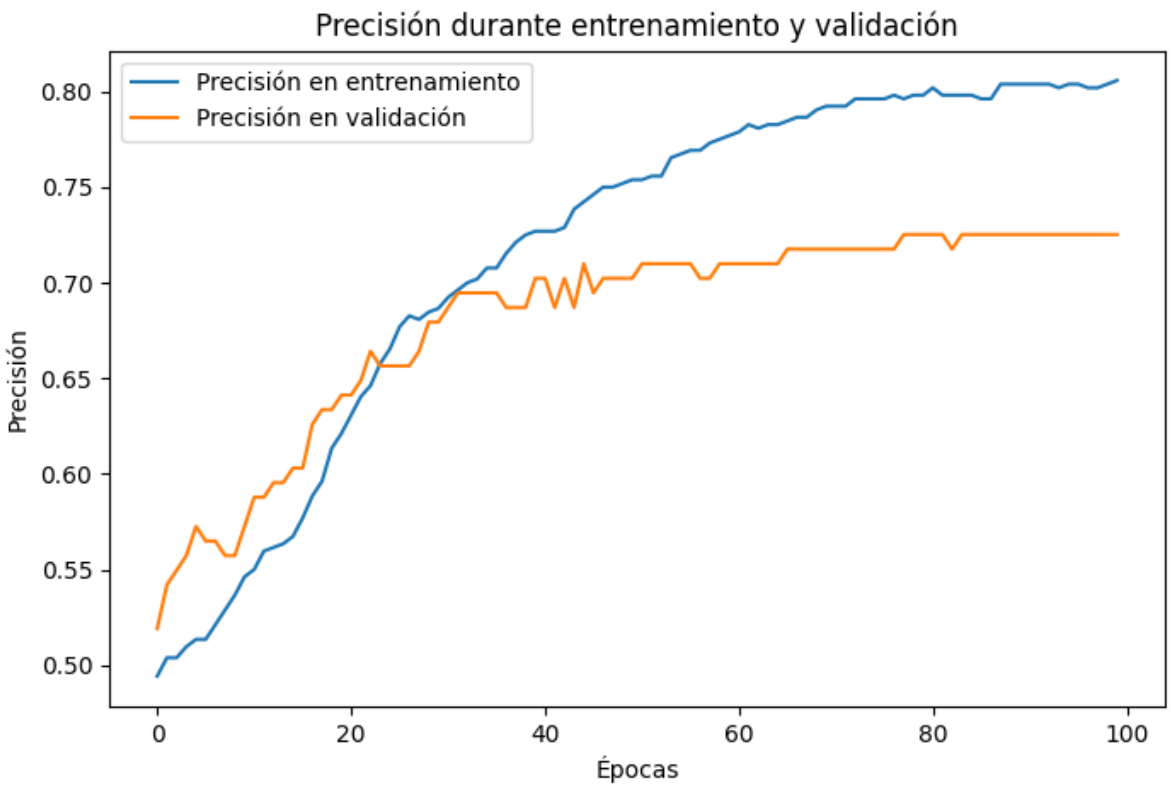
Los resultados demuestran que el accuracy inicia aproximadamente en valores de 0.7 y empieza a aumentar poco a poco hasta llegar a valores cercanos a 0.8 en el entrenamiento, mientras que en validación comienza alrededor de 0.7 y sube gradualmente hasta aproximadamente 0.74. Se puede decir que al usar PCA, se comienza con una accuracy más alta, a cambio de no mejorar tanto a través del entrenamiento en comparación.

El modelo sí está aprendiendo durante las épocas, ya que mejora su capacidad de clasificación y reduce su error.

6. Se grafican los valores de entrenamiento y los de validación.

In [394…

```
plt.figure(figsize=(8,5))
plt.plot(history2.history['accuracy'], label='Precisión en entrenamiento')
plt.plot(history2.history['val_accuracy'], label='Precisión en validación')
plt.legend()
plt.title("Precisión durante entrenamiento y validación")
plt.xlabel("Épocas")
plt.ylabel("Precisión")
plt.show()
```

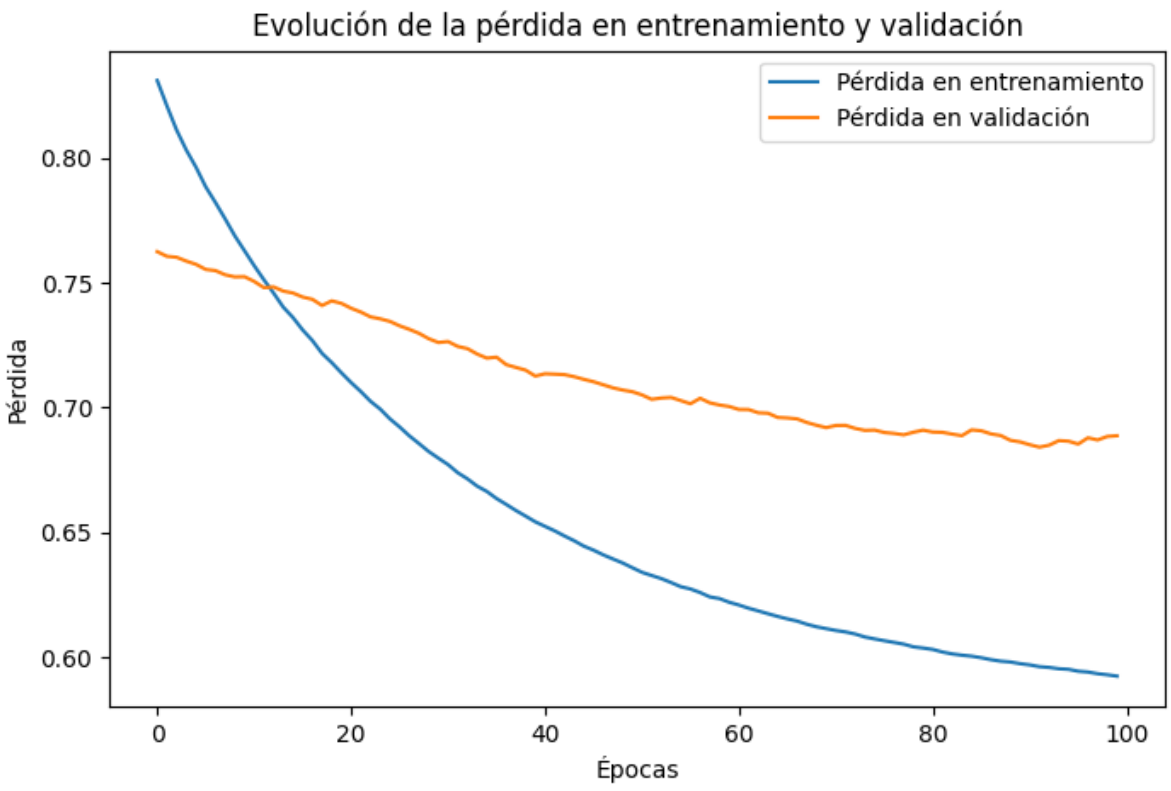


Podemos observar leves diferencias en comparación al primer modelo. La precisión de entrenamiento comienza con una pendiente más pronunciada, mostrando que a pesar de que se empeiza en un valor base más bajo, el aprendizaje del modelo es más veloz.

Ambas curvas aumentan poco y se mantienen relativamente cercanas, mostrando que se está mejorando el desempeño de manera generalizada. Sin embargo, alrededor de la época 38. Esto es posiblemente un problema de overfitting, donde se deja de generalizar bien, o un problema donde el modelo no es lo suficientemente complejo como para aprender los patrones en los datos apropiadamente, siendo que los datos ahora son componentes principales.

In [395…

```
plt.figure(figsize=(8,5))
plt.plot(history2.history['loss'], label='Pérdida en entrenamiento')
plt.plot(history2.history['val_loss'], label='Pérdida en validación')
plt.legend()
plt.title("Evolución de la pérdida en entrenamiento y validación")
plt.xlabel("Épocas")
plt.ylabel("Pérdida")
plt.show()
```



Podemos observar que, al usar PCA, el aprendizaje del modelo es mucho más veloz, alcanzando precisiones de validación a un nivel muy similar que el primer modelo, en la mitad de las épocas. Sin embargo, la mejoría de la precisión de validación, tanto del primer modelo como el segundo, deja de subir en un momento similar.

Es decir, ambos modelos tienen desempeño bueno, pero se podría decir que la mayor contirbución que dio PCA al segundo modelo fue simplificar el entrenamiento a través de la reducción de dimensionalidad. Debido a esto, el modelo pudo alcanzar las mismas altas precisiones que el primer modelo, en mucho menos tiempo.

Ya que estos recursos de tiempo se han "liberado", se podrían usar diferentes técnicas como otpimizaciones de parámetros para terminar con un modelo más preciso que utiliza una cantidad de recursos temporales y computacionales similares. Este es un gran beneficio por parte del uso de PCA. Sin embargo, para los propósitos de este ejercicio, podemos conformarnos con la mejoría del proceso ya presente.

In [396…

```
y_pred_prob = modelo.predict(X_test)
y_pred = (y_pred_prob > 0.5).astype(int)
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
```

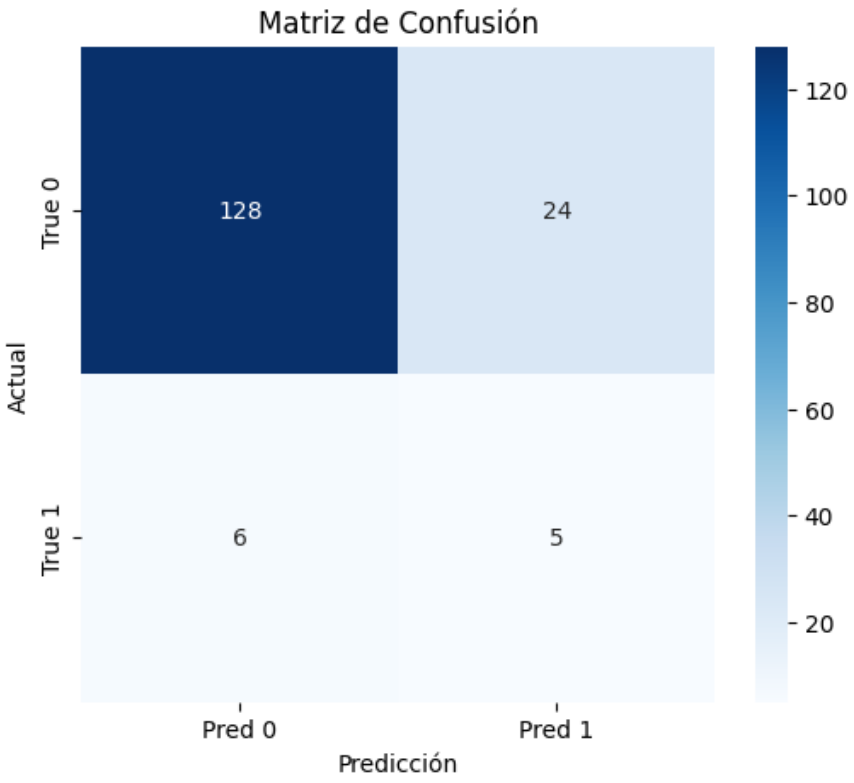
In [397…

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
```

```python
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Pred 0','Pred 1'],
    yticklabels=['True 0','True 1']
)

plt.xlabel('Predicción')
plt.ylabel('Actual')
plt.title('Matriz de Confusión')
plt.show()
```



La matriz nos muestra que los mayores problemas de precisión ocurren al momento donde la respuesta correcta sería un 0, pero el modelo predice un 1. Esto podría interpretarse como un enfoque en minimizar falsos negativos, como se mencionó anteriormente. De nuevo, representa una menor pérdida el "asustar" a un paciente con una diagnosis falsa que luego puede ser analizada a profundidad para corregirla, en lugar que a permitir que un paciente enfermo piense que está sano.

In [398…

```
# Calcular métricas.
precision2 = precision_score(y_test, y_pred)
recall2 = recall_score(y_test, y_pred)

# Extraer valores para matriz.
tn, fp, fn, tp = cm.ravel()
especificidad2 = tn / (tn + fp)

# Calcular F1 score.
f1_2 = f1_score(y_test, y_pred)

print("Precisión:", precision2)
print("Sensibilidad (Recall):", recall2)
print("Especificidad:", especificidad2)
print("Puntaje F1:", f1_2)
```

```
Precisión: 0.1724137931034483
Sensibilidad (Recall): 0.45454545454545453
Especificidad: 0.8421052631578947
Puntaje F1: 0.25
```

In [399…

```
modelo.save("single_neuron_antes_de_pca.keras")
```

In [400…

```
modelo_cargado = tf.keras.models.load_model("single_neuron_antes_de_pca.keras")
```

Comparación

In [401…

```
# Definir tabla y columnas.
tabla_metricas = pd.DataFrame({
    "Modelo 1": [precision, recall, especificidad, f1],
    "Modelo PCA": [precision2, recall2, especificidad2, f1_2]
},
# Definir nombres de filas.
index=[
    "Precisión",
    "Sensibilidad (Recall)",
    "Especificidad",
    "Puntaje F1"
])

print(tabla_metricas)
```

```
                      Modelo 1  Modelo PCA
Precisión             0.225806    0.172414
Sensibilidad (Recall) 0.636364    0.454545
Especificidad         0.842105    0.842105
Puntaje F1            0.333333    0.250000
```

Podemos observar que el modelo de PCA no solo alcanza buenas métricas en menor tiempo, si no que además cuenta con mejores métricas en todos los aspectos: mejor precisión, recall, especificidad, y puntaje F1.

Todas estas mejorías se lograron con un comportamiento de aprendizaje mucho más veloz, y la estabilidad de entrenamiento se mantuvo a un nivel similar, sin empeoro evidente.

La pérdida de entrenamiento se amntiene similarmente "suave" en ambos casos, sin saltos fuertes. Para ambos modelos, la pérdida de validación se aplana en valores similares. es decir, no se muestra empeoro considerable en el entrenamiento en cuanto a los resultados en validación, aunque existen áreas de oportunidad para lograr que el modelo generalize mejor, con o sin PCA.

Debido a estas métricas, se puede concluir que PCA fue extremadamente beneficioso, mejorando la manera en que el aprendizaje es realizado, obteniendo mejores métricas en todas las áreas. La reducción de la dimensionalidad gracias a PCA llevó a un mejor modelo, tomando menos épocas. No hubo un trade-off entre la dimensionalidad y la performance. Al contrario, la simplificación del entrenamiento permitió llegar a mejor desempeño en las mismas 100 épocas. El único trade-off debatible del uso de PCA en este caso, sería la pérdida de interpretabilidad que sucede al usar componentes principales.

En general, si esta interpretabilidad no fuese una preocupación mayor, sería mejor desplegar el modelo PCA, al ser más rápido y además más preciso.

Conclusiones

Einer:

Esta actividad fue una demostración excelente de los beneficios posibles de PCA, tanto en el área de simplificación del proceso de entrenamiento como en sus posibles beneficios. Fue interesante hacer uso de una sola neurona para la predicción de algo tan complejo como lo sería una diagnósis médica, aunque por supuesto no se tiene una precisión perfecta. Existen varias limitaciones del estudio, por supuesto, pues el enfoque se hizo en el uso de single-neuron networks y PCA, no específicamente en la optimización perfecta de los modelos. Por ello, la precisión es un área de oportunidad. Sin embargo, con la aplicación actual de esta tarea, se puede observar excelentemente el efecto de PCA y el funcionamiento de las single-neuron networks, por lo cual me parece un desarrollo satisfactorio para el aprendizaje planteado para esta actividad.

Pasos siguientes para mejorar esta precisión incluirían data engineering con o sin PCA, optimización de hiperparámetros, y si buscamos alejarnos de las single-neuron networks, podríamos epxlorar diferentes configuraciones de la network que quizás puedan captar los patrones en los datos con más eficacia.

Github: https://github.com/einerbarba-u/04-single-neuron.git

Renata:

Se puede concluir que esta actividad nos ha permitido comprender como un modelo de una sola nuerona funciona y como se puede aplicar a un problema de clasificacación siendo en este caso dentro de un contexto médico. A través de la actividad se ha aprendido la importancia de seguir un proceso de paso a paso empezando por el preprocesamiento de los datos, continuando con la limpieza, la imputación de valores faltantes y la creación del modelo. El uso de un modelo de una sola neurona demostró que sí existen varias limitaciones debido que en este caso los patrones probablemente eran muy complejos para poder representar las relaciones entre las variables por lo cual le favorecería un modelo de más de una sola neurona. A pesar de esto, se demuestra que el modelo funciona solamente que no de la manera ideal. Por otro lado, al aplicar PCA en este caso se redujo la dimensionalidad logrando un resultado bastante favorable ya que mostró resultados muy similares pero con una cantidad menor de dimensiones.

En el futuro se podría buscar intentar otro tipo de técnicas para mejorar el mode. Por ejemplo, uno de los problemas fue el desbalance entre las clases entonces se podrían usar otras técnicas para mitigar el efecto lo más posible. También se podrían intentar ajustar los hiperparámetros o usar arquitecturas de redes neuronales que sean más profundas que quizás sean más favorables para este dataset.

Github: https://github.com/RenataGM/Single-NeuronNetworkWithAndWithoutPCA.git

Referencias

Arunachalam, B. (2023, septiembre 21). Binary classification with TensorFlow tutorial. freeCodeCamp. https://www.freecodecamp.org/news/binary-classification-made-simple-with-tensorflow/

García Morales, R., & Villareal Alanís, E. M. (2025, noviembre 7). Intro Redes Neuronales [Notas de clase]. Google Colab. https://colab.research.google.com/drive/1suGLRI9sKRz6pfuJaP8RzmlDWgce1RB7?authuser=1

GeeksforGeeks. (2025, agosto 11). Single Layer Perceptron in TensorFlow. GeeksforGeeks. https://www.geeksforgeeks.org/python/single-layer-perceptron-in-tensorflow/